

INFORMATIQUE • 3ÈME

INFORMATIQUE – CLASSE DE 3ÈME

## LEÇON N14

## UTILISATION DES STRUCTURES DE CONTRÔLE

DURÉE CONSEILLÉE : 3 SÉANCES (FICHE DE PROGRESSION HARMONISÉE)

## DANS CETTE LEÇON

Cette leçon introduit l'écriture d'algorithmes simples et l'utilisation des structures alternatives (si... alors... sinon) et itératives (tant que, pour) pour résoudre des problèmes concrets. Elle présente également l'algorithme, représentation graphique normalisée d'un algorithme.

- À la fin de cette leçon, je sais définir un algorithme et identifier ses constituants (variables, instructions, entrées, sorties)
- À la fin de cette leçon, je sais écrire un algorithme simple en pseudo-code pour résoudre un problème de la vie courante
- À la fin de cette leçon, je sais utiliser une structure alternative simple (si... alors) et complète (si... alors... sinon)
- À la fin de cette leçon, je sais utiliser une structure itérative « tant que » et vérifier qu'elle se termine
- À la fin de cette leçon, je sais utiliser une structure itérative « pour » pour répéter un nombre connu de fois
- À la fin de cette leçon, je sais représenter un algorithme par un algorithme en respectant les symboles normalisés



## TÉLÉCHARGE ONBUCH

Gratuit • Google Play  
Tuteur IA, annales,  
concours, résultats



## PASSE À ONBUCH+

Tous les cours signés, Tuteur IA  
illimité, fiches PDF — onglet  
OnBuch+ de l'app

## CE COURS SIGNÉ CONTIENT

## COURS

complet, illustré, pas à pas

## MÉTHODES

et exercices résolus

## EXERCICES

type examen + corrigés

## FICHE BILAN

l'essentiel à retenir



Signé • OnBuch+

Ludovic A.

CONFORME AU PROGRAMME OFFICIEL MINESEC

OnBuch+

**SOMMAIRE**

1. Notion d'algorithme et pseudo-code
2. Les structures alternatives
3. Les structures itératives : la boucle « tant que »
4. Les structures itératives : la boucle « pour »
5. L'algorithme : représentation graphique
6. Résolution de problèmes concrets et justification
7. Activité d'intégration
8. Méthodes et exercices résolus
9. Exercices
10. Corrigés des exercices
11. Fiche bilan

**OBJECTIFS DE LA LEÇON**

- À la fin de cette leçon, je sais définir un algorithme et identifier ses constituants (variables, instructions, entrées, sorties)
- À la fin de cette leçon, je sais écrire un algorithme simple en pseudo-code pour résoudre un problème de la vie courante
- À la fin de cette leçon, je sais utiliser une structure alternative simple (si... alors) et complète (si... alors... sinon)
- À la fin de cette leçon, je sais utiliser une structure itérative « tant que » et vérifier qu'elle se termine
- À la fin de cette leçon, je sais utiliser une structure itérative « pour » pour répéter un nombre connu de fois
- À la fin de cette leçon, je sais représenter un algorithme par un algorithme en respectant les symboles normalisés
- À la fin de cette leçon, je sais dérouler (tracer) un algorithme à la main pour vérifier son fonctionnement
- À la fin de cette leçon, je sais rédiger et justifier la démarche suivie pour résoudre un problème concret à l'aide d'un algorithme

**PRÉREQUIS**

- Notion de variable et d'affectation (début d'année de 3ème)

- Opérations arithmétiques et comparaisons ( $<$ ,  $>$ ,  $=$ ,  $\leq$ ,  $\geq$ ,  $\neq$ )
- Expressions logiques simples : conditions vraies ou fausses
- Résolution de problèmes numériques simples (calcul de moyenne, de prix, de périmètre)

## 1 NOTION D'ALGORITHME ET PSEUDO-CODE

**Situation d'accroche.** Maman Aïcha vend des beignets au marché de Mokolo, à Yaoundé. Chaque matin, sans même y réfléchir, elle répète les mêmes gestes : tant qu'il reste de la pâte, elle forme un beignet et le fait frire ; si un client achète pour plus de 500 FCFA, elle offre un beignet en plus ; sinon, elle dit simplement merci. Le soir, elle compte sa recette : elle additionne les ventes, soustrait le prix de la farine et de l'huile, et obtient son bénéfice.

Sans le savoir, Maman Aïcha exécute un **algorithme** : une suite d'instructions précises, dans un ordre bien défini, avec des conditions (« si le client achète pour plus de 500 FCFA... ») et des répétitions (« tant qu'il reste de la pâte... »).

**Problématique.** Comment écrire de telles instructions de façon précise et sans ambiguïté, pour qu'une machine (ou une personne) les exécute sans erreur ? C'est tout l'objet de cette leçon. Dans cette première section, nous découvrons ce qu'est un algorithme, comment on le décrit en **pseudo-code**, et comment vérifier qu'il fonctionne grâce à la **trace d'exécution**.

### 1.1 Qu'est-ce qu'un algorithme ?

**Intuition.** Imagine que tu dois expliquer à un petit frère comment préparer un thé : « mets de l'eau dans la bouilloire, fais chauffer, mets le thé dans la tasse, verse l'eau chaude, attends 3 minutes ». Tu viens de donner un algorithme ! Chaque étape est claire, les étapes sont dans le bon ordre (on ne verse pas l'eau avant de l'avoir chauffée), et au bout d'un nombre fini d'étapes, le résultat est obtenu : le thé est prêt.

En informatique, c'est exactement la même idée : avant de demander à un ordinateur de faire un calcul, on décrit d'abord, en langage clair, la suite des opérations à effectuer.

#### DÉFINITION Algorithme

Un **algorithme** est une suite finie et ordonnée d'instructions permettant de résoudre un problème ou d'obtenir un résultat à partir de données fournies.

Décomposons cette définition :

- **suite finie** : l'algorithme s'arrête toujours après un nombre fini d'étapes ;
- **ordonnée** : les instructions s'exécutent dans un ordre précis, qu'on ne peut pas changer n'importe comment ;
- **instructions** : chaque étape est un ordre clair et réalisable (calculer, lire, afficher, comparer...);
- **résoudre un problème** : l'algorithme part de données et produit un résultat.

#### LESAVAIS-TU ? D'où vient le mot « algorithme » ?

Le mot « algorithme » vient du nom du mathématicien perse Al-Khwarizmi, qui vécut au IX<sup>e</sup>

siècle à Bagdad. Dans ses ouvrages, il décrivait des méthodes de calcul étape par étape (par exemple pour résoudre des équations). Son nom, latinisé en Algoritmi, a donné le mot « algorithme ». C'est aussi de son livre sur le calcul avec les chiffres indiens que vient le mot « algèbre » (al-jabr). Plus de mille ans plus tard, chaque application de ton téléphone exécute des algorithmes inspirés de cette idée !

## 1.2 Entrées, traitements, sorties : la « machine à transformer »

Tout algorithme peut être vu comme une « machine à transformer » : on lui donne des **entrées**, il effectue des **traitements**, et il produit des **sorties**.

### DÉFINITION Entrées, traitements, sorties

- Les **entrées** sont les données fournies à l'algorithme avant son exécution (par exemple : les deux notes d'un élève).
- Les **traitements** sont les opérations effectuées sur ces données (par exemple : additionner les notes, diviser par 2).
- Les **sorties** sont les résultats produits et affichés (par exemple : la moyenne de l'élève).

Un bon algorithme possède trois qualités essentielles :

### PROPRIÉTÉ Qualités d'un bon algorithme

1. **Finitude** : l'algorithme doit se terminer après un nombre fini d'étapes, quelles que soient les entrées.
2. **Clarté (non-ambiguïté)** : chaque instruction doit avoir un seul sens possible. « Ajoute un peu de sel » n'est pas une instruction claire ; « ajoute 5 g de sel » l'est.
3. **Efficacité** : l'algorithme doit produire le bon résultat, et de préférence sans étapes inutiles.

### EXEMPLE Des algorithmes dans la vie courante

- **Recette de cuisine** : entrées = les ingrédients ; traitements = laver, couper, mélanger, cuire ; sortie = le plat (par exemple un eru bien préparé).
- **Itinéraire** : entrées = le point de départ (Carrefour Wada) et la destination (le collège) ;

traitements = la suite des directions ; sortie = l'arrivée au collège.

- Retrait d'argent au guichet mobile (OM/MoMo) : entrées = ton numéro, le montant, ton code secret ; traitements = vérifier le code, vérifier que le solde est suffisant, débiter le compte ; sorties = l'argent remis et le message de confirmation. Remarque l'ordre : on vérifie le code avant de donner l'argent !

### 1.3 Les variables : les « boîtes » de l'algorithme

**Intuition.** Pour calculer une moyenne, l'algorithme doit mémoriser les deux notes, puis la somme, puis la moyenne. Où stocke-t-on ces valeurs ? Dans des **variables**.

#### DÉFINITION Variable

Une **variable** est une zone de la mémoire, repérée par un nom (appelé identifiant), qui contient une valeur pouvant changer au cours de l'exécution de l'algorithme. On peut imaginer une variable comme une boîte étiquetée : le nom est écrit sur la boîte, la valeur est rangée dedans.

Par exemple, la variable nommée `note1` peut contenir la valeur 12. Plus tard, on pourra ranger 14 dans cette même boîte : l'ancienne valeur est alors remplacée.

Quelques règles pour nommer les variables :

- le nom commence par une lettre et ne contient ni espace ni accent : on écrit `note1`, `moyenne`, `prixTotal` ;
- on choisit des noms qui ont un sens : `moyenne` est bien plus clair que `x` ou `truc`.

### 1.4 Le pseudo-code : un langage pour écrire les algorithmes

**Intuition.** Un ordinateur ne comprend que des langages de programmation très stricts. Mais avant de programmer, il faut d'abord penser la solution. On utilise alors le **pseudo-code** : un langage simplifié, en français, qui décrit l'algorithme sans se soucier des détails techniques d'un vrai langage. Le pseudo-code est compréhensible par tout le monde et pourra ensuite être traduit dans n'importe quel langage de programmation.

#### MÉTHODE Structure générale d'un algorithme en pseudo-code

Tout algorithme écrit en pseudo-code respecte la structure suivante, en quatre parties :

1. La ligne **Algorithme** suivie du nom de l'algorithme ;
2. La déclaration des variables : on annonce toutes les « boîtes » dont on aura besoin, avec leur type (nombre, texte...) ;

3. Le corps, compris entre `Début` et `Fin` : c'est là qu'on écrit les instructions, dans l'ordre où elles seront exécutées ;
4. La ligne `Fin` qui marque l'arrêt de l'algorithme.

**PROPRIÉTÉ** Syntaxe générale

```
Algorithme nom_de_l_algorithme
Variables
    nom1 : type
    nom2 : type
Début
    instruction 1
    instruction 2
    ...
Fin
```

Les types usuels en classe de 3ème sont : `Entier` (nombre sans virgule), `Réel` (nombre à virgule), `Chaîne` (texte).

## 1.5 Les instructions de base : lire, écrire, affecter

Trois instructions suffisent pour écrire nos premiers algorithmes.

**1. L'instruction `Lire` : la saisie des données.** Elle permet à l'utilisateur d'entrer une valeur au clavier, qui sera rangée dans une variable. C'est l'instruction des entrées.

`Lire note1` signifie : « demande une valeur à l'utilisateur et range-la dans la variable `note1` ».

**2. L'instruction `Écrire` : l'affichage des résultats.** Elle affiche à l'écran un message ou la valeur d'une variable. C'est l'instruction des sorties.

`Écrire "La moyenne est : ", moyenne` affiche le texte puis la valeur contenue dans `moyenne`.

**3. L'affectation : ranger une valeur dans une variable.** On utilise le symbole  $\leftarrow$  (une flèche vers la gauche) :

$$\text{moyenne} \leftarrow (\text{note1} + \text{note2}) / 2$$

Cette instruction se lit : « calcule la valeur de  $(\text{note1} + \text{note2}) / 2$ , puis range le résultat dans la variable `moyenne` ». On évalue d'abord ce qui est à droite de la flèche, puis on range le résultat dans la variable à gauche.

**ATTENTION** Affectation ou égalité mathématique ?

L'erreur la plus fréquente des débutants est de confondre l'affectation  $x \leftarrow 5$  avec l'égalité mathématique  $x = 5$ .

- En mathématiques,  $x = 5$  est une affirmation : «  $x$  vaut 5 ». On peut écrire  $x = 5$  ou  $5 = x$ , c'est pareil.
- En algorithmique,  $x \leftarrow 5$  est un ordre : « range la valeur 5 dans la boîte  $x$  ». L'ordre compte :  $5 \leftarrow x$  n'a aucun sens, car on ne peut pas ranger une valeur dans le nombre 5!

Conséquence importante : l'instruction  $x \leftarrow x + 1$  est parfaitement correcte en algorithmique (on prend l'ancienne valeur de  $x$ , on ajoute 1, et on range le résultat dans  $x$ ), alors que  $x = x + 1$  est impossible en mathématiques!

## 1.6 Premier algorithme complet : la moyenne de deux notes

**Problème.** Le professeur d'informatique du collège veut un algorithme qui demande les deux notes d'un élève (sur 20), calcule la moyenne et l'affiche.

**Analyse du problème** (toujours avant d'écrire!) :

- **Entrées** : deux notes, que nous appellerons `note1` et `note2`;
- **Traitement** : calculer  $\text{moyenne} = \frac{\text{note1} + \text{note2}}{2}$ ;
- **Sortie** : afficher la moyenne.

### EXEMPLE Algorithme « Moyenne de deux notes »

```

Algorithme MoyenneDeuxNotes
Variables
    note1, note2 : Reel    // les deux notes saisies
    moyenne : Reel        // le resultat calcule
Debut
    Ecrire "Entrez la premiere note : "
    Lire note1             // saisie de la 1re note
    Ecrire "Entrez la deuxieme note : "
    Lire note2             // saisie de la 2e note
    moyenne  $\leftarrow$  (note1 + note2) / 2 // calcul de la moyenne
    Ecrire "La moyenne est : ", moyenne // affichage du resultat
Fin
  
```

Lisons cet algorithme ligne par ligne : on affiche d'abord un message pour guider l'utilisateur, on lit la première note, puis la deuxième ; ensuite on calcule la moyenne par affectation ; enfin on affiche le résultat. L'ordre est essentiel : on ne peut pas calculer la moyenne avant d'avoir lu les deux notes !

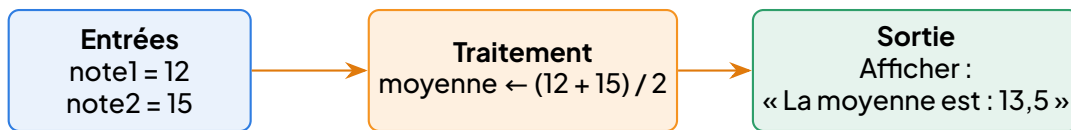


Schéma « Entrées → Traitement → Sorties » pour l'algorithme de la moyenne.

## 1.7 La trace d'exécution : vérifier son algorithme pas à pas

**Intuition.** Comment être sûr qu'un algorithme est correct avant de le faire exécuter par un ordinateur? On joue soi-même le rôle de la machine : on exécute les instructions une par une, à la main, et on note dans un tableau la valeur de chaque variable après chaque étape. Ce tableau s'appelle la **trace d'exécution**. C'est un outil formidable pour détecter les erreurs.

### EXERCICE RÉSOLU 1 Trace complète de l'algorithme de la moyenne

**Énoncé.** On exécute l'algorithme `MoyenneDeuxNotes` avec les valeurs saisies `note1 = 12` et `note2 = 15`. Donner la trace d'exécution complète et le résultat affiché.

#### SOLUTION

Solution détaillée. On suit les instructions dans l'ordre et on note le contenu de chaque variable après chaque étape (le tiret « - » signifie que la variable n'a pas encore de valeur) :

| Étape | Instruction exécutée                             | note1 | note2 | moyenne |
|-------|--------------------------------------------------|-------|-------|---------|
| 1     | Lire <code>note1</code> (l'utilisateur tape 12)  | 12    | -     | -       |
| 2     | Lire <code>note2</code> (l'utilisateur tape 15)  | 12    | 15    | -       |
| 3     | <code>moyenne ← (12 + 15) / 2</code>             | 12    | 15    | 13,5    |
| 4     | Écrire "La moyenne est : ", <code>moyenne</code> | 12    | 15    | 13,5    |

Détail du calcul de l'étape 3 :

$$\text{moyenne} = \frac{\text{note1} + \text{note2}}{2} = \frac{12 + 15}{2} = \frac{27}{2} = 13,5$$

Résultat affiché à l'écran : La moyenne est : 13,5

L'algorithme se termine (finitude respectée), chaque instruction était claire, et le résultat est correct : notre algorithme est validé.

### ATTENTION Les pièges classiques du débutant

- Utiliser une variable non déclarée : toute variable doit être annoncée dans la partie

Variables avant d'être utilisée.

- Calculer avant de lire : écrire  $\text{moyenne} \leftarrow (\text{note1} + \text{note2}) / 2$  avant les instructions Lire donne un résultat faux, car les boîtes sont encore vides.
- Oublier les parenthèses :  $(\text{note1} + \text{note2}) / 2$  et  $\text{note1} + \text{note2} / 2$  ne donnent pas le même résultat ! Sans parenthèses, la division est faite en priorité :  $12 + 15/2 = 19,5$  au lieu de 13,5.
- Oublier d'afficher le résultat : un calcul qui n'est jamais affiché ne sert à rien pour l'utilisateur.

### EXERCICE 1 À toi de jouer !

1. Maman Aïcha vend chaque beignet à 100 FCFA. Écris en pseudo-code un algorithme Recette qui demande le nombre de beignets vendus dans la journée, calcule la recette et l'affiche.
2. Donne la trace d'exécution de ton algorithme pour 230 beignets vendus.
3. Que se passe-t-il si l'on écrit  $\text{recette} \leftarrow \text{nombre} * 100$  avant d'avoir lu la variable nombre ? Explique.

### CORRIGÉ Exercice « À toi de jouer ! »

#### 1. Algorithme Recette :

Algorithme Recette

Variables

nombre : Entier // nombre de beignets vendus  
recette : Entier // recette de la journée en FCFA

Debut

Ecrire "Entrez le nombre de beignets vendus : "

Lire nombre

recette  $\leftarrow$  nombre \* 100 // chaque beignet coute 100 FCFA

Ecrire "La recette du jour est : ", recette, " FCFA"

Fin

#### 2. Trace d'exécution pour nombre = 230 :

| Étape | Instruction exécutée                                 | nombre | recette |
|-------|------------------------------------------------------|--------|---------|
| 1     | Lire nombre (l'utilisateur tape 230)                 | 230    | -       |
| 2     | $\text{recette} \leftarrow 230 * 100$                | 230    | 23 000  |
| 3     | Écrire "La recette du jour est : ", recette, " FCFA" | 230    | 23 000  |

Résultat affiché : La recette du jour est : 23000 FCFA.

3. Si l'on écrit `recette ← nombre * 100` avant le Lire nombre, la variable nombre ne contient encore aucune valeur : le calcul est impossible (ou donne un résultat imprévisible). C'est une violation de l'ordre logique des instructions : on doit toujours lire les entrées avant de les utiliser dans un traitement.

**À RETENIR****L'essentiel de la section**

- Un **algorithme** est une suite finie et ordonnée d'instructions qui résout un problème : il part d'entrées, effectue des traitements et produit des sorties.
- Une **variable** est une boîte nommée qui contient une valeur modifiable.
- En **pseudo-code**, un algorithme s'écrit avec la structure Algorithme / Variables / Début / Fin.
- Les trois instructions de base sont : Lire (saisie), Écrire (affichage) et l'affectation  $\leftarrow$  (ranger une valeur calculée dans une variable).
- L'affectation  $x \leftarrow 5$  n'est pas une égalité mathématique : c'est un ordre, et le sens de la flèche compte.
- La **trace d'exécution** est un tableau qui suit les valeurs des variables ligne par ligne : c'est l'outil pour vérifier qu'un algorithme est juste.

Dans la prochaine section, nous verrons comment un algorithme peut prendre des décisions grâce aux structures alternatives (« si... alors... sinon »), comme Maman Aïcha qui offre un beignet seulement si l'achat dépasse 500 FCFA.

**2****LES STRUCTURES ALTERNATIVES**

Dans la section précédente, nous avons découvert ce qu'est un **algorithme** : une suite finie et ordonnée d'instructions, écrite en **pseudo-code**, qui permet de résoudre un problème. Jusqu'ici, nos algorithmes exécutaient leurs instructions les unes après les autres, toujours dans le même ordre : on parle d'exécution séquentielle. Mais la vie n'est pas une ligne droite ! Le professeur n'affiche pas le même message pour tous les élèves : il affiche « Admis » pour l'un, « Refusé » pour l'autre. La caissière de la boutique n'applique pas toujours la même remise. Bref : un bon algorithme doit savoir prendre des décisions. C'est exactement le rôle des **structures alternatives**, que nous allons étudier dans cette section.

**2.1 La notion de condition : une question qui n'a que deux réponses**

Avant de décider, il faut pouvoir tester. Imagine la scène suivante : à la fin du BEPC blanc, le professeur principal regarde la moyenne de chaque élève. Pour chacun, il se pose une seule question : « la moyenne est-elle supérieure ou égale à 10 ? ». À cette question, il n'existe que

deux réponses possibles : oui ou non. Une telle question s'appelle une **condition**.

**DÉFINITION** Condition

Une **condition** est une expression qui ne peut prendre que deux valeurs : VRAI ou FAUX. On dit aussi que c'est une expression booléenne.

**EXEMPLE** Des conditions de la vie courante

- «  $\text{note} \geq 10$  » : VRAI si l'élève a 12, FAUX s'il a 8 ;
- «  $\text{solde} \geq \text{montant du retrait}$  » : VRAI si le compte contient assez d'argent ;
- «  $\text{âge} < 18$  » : VRAI pour un élève de 3ème de 14 ans ;
- «  $\text{mot de passe} = \text{"1234"}$  » : VRAI seulement si la saisie est exacte.

Pour écrire des conditions, on utilise les **opérateurs de comparaison**. Ils comparent deux valeurs (deux nombres, un nombre et une variable, etc.) et le résultat est VRAI ou FAUX.

| Opérateur | Signification               | Exemple (avec $\text{note} = 12$ ) |
|-----------|-----------------------------|------------------------------------|
| =         | est égal à                  | $\text{note} = 12$ est VRAI        |
| ≠         | est différent de            | $\text{note} \neq 10$ est VRAI     |
| <         | est strictement inférieur à | $\text{note} < 10$ est FAUX        |
| >         | est strictement supérieur à | $\text{note} > 10$ est VRAI        |
| ≤         | est inférieur ou égal à     | $\text{note} \leq 12$ est VRAI     |
| ≥         | est supérieur ou égal à     | $\text{note} \geq 10$ est VRAI     |

Il arrive qu'une seule comparaison ne suffise pas. Par exemple, la mention « Bien » au BEPC exige une moyenne à la fois supérieure ou égale à 14 et strictement inférieure à 16. On combine alors plusieurs conditions grâce aux **connecteurs logiques** :

**PROPRIÉTÉ** Connecteurs logiques ET et OU

- $\text{condition1 ET condition2}$  est VRAI seulement si les deux conditions sont vraies en même temps. Exemple :  $(\text{moyenne} \geq 14) \text{ ET } (\text{moyenne} < 16)$ .
- $\text{condition1 OU condition2}$  est VRAI dès qu'au moins une des deux conditions est vraie. Exemple :  $(\text{jour} = \text{"samedi"}) \text{ OU } (\text{jour} = \text{"dimanche"})$ .

## 2.2 La structure alternative simple : Si ... Alors ... FinSi

Revenons à notre boutique de quartier à Yaoundé. Le commerçant a décidé : « si un client achète

pour plus de 10000 FCFA, je lui offre un bon de réduction ». Remarque bien : si le client achète pour 5000 FCFA, le commerçant ne fait rien de spécial. Il n'y a qu'un seul cas où une action est déclenchée. C'est la situation typique de l'alternative simple.

**DÉFINITION** Structure alternative simple

La **structure alternative simple** exécute un bloc d'instructions uniquement si la condition est vraie. Si la condition est fausse, on ne fait rien et on passe directement à la suite de l'algorithme.

**PROPRIÉTÉ** Syntaxe de l'alternative simple

```
Si condition Alors
    instructions
FinSi
```

La condition est évaluée : si elle est VRAIE, les instructions sont exécutées ; si elle est FAUSSE, elles sont ignorées. Dans les deux cas, l'algorithme continue après le FinSi.

**EXEMPLE** Féliciter les bons élèves

```
Algorithme Felicitations
Variable note : Reel
Debut
    Ecrire("Entrez la note : ")
    Lire(note)
    Si note >= 16 Alors
        Ecrire("Félicitations, excellent travail !")
    FinSi
    Ecrire("Fin du programme.")
Fin
```

Si l'élève saisit 17, le message de félicitations s'affiche. S'il saisit 12, le programme affiche directement « Fin du programme. » sans autre message.

## 2.3 La structure alternative complète : Si ... Alors ... Sinon ... FinSi

Très souvent, on ne veut pas « faire ou ne rien faire », mais choisir entre deux actions. Le professeur ne se contente pas d'afficher « Admis » : il doit afficher « Refusé » pour les autres. Il y a deux cas et ils s'excluent mutuellement. C'est le rôle de l'alternative complète.

**DÉFINITION** Structure alternative complète

La **structure alternative complète** exécute un premier bloc d'instructions si la condition est vraie, et un autre bloc si elle est fausse. L'un des deux blocs est toujours exécuté, mais jamais les deux.

**PROPRIÉTÉ** Syntaxe de l'alternative complète

```
Si condition Alors
    instructions1
Sinon
    instructions2
FinSi
```

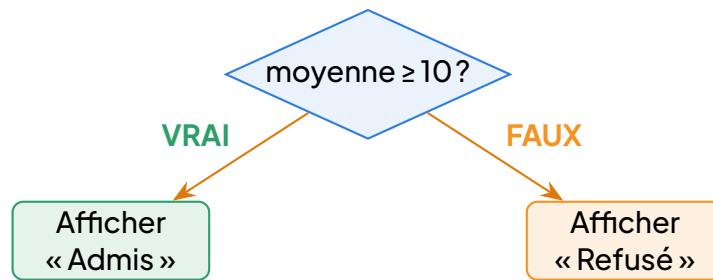
Si la condition est VRAIE, seul `instructions1` est exécuté; si elle est FAUSSE, seul `instructions2` est exécuté. Ensuite, l'algorithme continue après le `FinSi`.

**EXEMPLE** Admis ou Refusé

```
Algorithme Resultat
Variable moyenne : Reel
Debut
    Ecrire("Entrez la moyenne : ")
    Lire(moyenne)
    Si moyenne >= 10 Alors
        Ecrire("Admis")
    Sinon
        Ecrire("Refuse")
    FinSi
Fin
```

Avec une moyenne de 11,5 le programme affiche « Admis »; avec 9,5 il affiche « Refusé ».

La figure suivante représente cette décision sous la forme d'un arbre de décision : on part de la condition, puis on suit la branche « VRAI » ou la branche « FAUX ».



Arbre de décision de l'alternative complète : la condition « moyenne  $\geq$  10 » oriente l'exécution vers l'une des deux branches, jamais les deux.

#### MÉTHODE Choisir entre alternative simple et alternative complète

1. Lis bien l'énoncé et repère la décision à prendre (la condition).
2. Pose-toi la question : « si la condition est fausse, doit-on faire quelque chose de précis ? »
3. Si non (on ne fait rien de spécial), utilise l'alternative simple : Si ...Alors ...FinSi.
4. Si oui (il y a une action pour le cas contraire), utilise l'alternative complète : Si ...Alors ...Sinon ...FinSi.
5. Vérifie enfin que chaque Si possède bien son FinSi.

#### EXERCICE RÉSOLU 2 La remise de 5 %

Énoncé. Une boutique de Douala accorde une remise de 5 % sur tout achat dont le montant dépasse 10000 FCFA. Écris un algorithme qui lit le montant de l'achat et affiche le prix à payer. Fais ensuite la trace (le déroulement pas à pas) pour un achat de 8000 FCFA puis pour un achat de 15000 FCFA.

#### SOLUTION

Solution. Si le montant dépasse 10000 FCFA, on calcule la remise (5 % du montant, c'est-à-dire montant  $\times 0,05$ ) et on la retire. Sinon, le prix reste inchangé : c'est une alternative complète.

```

Algorithme PrixAPayer
Variable montant, prix : Reel
Debut
  Ecrire("Entrez le montant de l'achat : ")
  Lire(montant)
  Si montant > 10000 Alors

```

```

    prix <- montant - montant * 0.05
Sinon
    prix <- montant
FinSi
Ecrire("Prix a payer : ", prix, " FCFA")
Fin

```

Trace pour montant = 8000 (tableau de trace) :

| Instruction        | Valeur / Action                      |
|--------------------|--------------------------------------|
| Lire(montant)      | 8000                                 |
| Si montant > 10000 | FAUX (8000 > 10000 est faux)         |
| Exécuter Sinon     | prix ← 8000                          |
| Ecrire(prix)       | Affiche « Prix a payer : 8000 FCFA » |

Trace pour montant = 15000 :

| Instruction        | Valeur / Action                       |
|--------------------|---------------------------------------|
| Lire(montant)      | 15000                                 |
| Si montant > 10000 | VRAI (15000 > 10000 est vrai)         |
| Exécuter Alors     | prix ← 15000 - 15000 × 0,05 = 14250   |
| Ecrire(prix)       | Affiche « Prix a payer : 14250 FCFA » |

## 2.4 Les alternatives imbriquées : plusieurs cas possibles

Que faire quand il y a plus de deux cas ? Au BEPC, on distingue par exemple quatre situations : Ajourné, mention Passable, mention Bien, mention Très Bien. Une seule alternative ne suffit plus. La solution consiste à placer une alternative dans le `Sinon` d'une autre : on parle d'**alternatives imbriquées**.

### PROPRIÉTÉ Syntaxe des alternatives imbriquées

```

Si condition1 Alors
    instructions1
Sinon
    Si condition2 Alors
        instructions2
    Sinon
        instructions3
    FinSi
FinSi

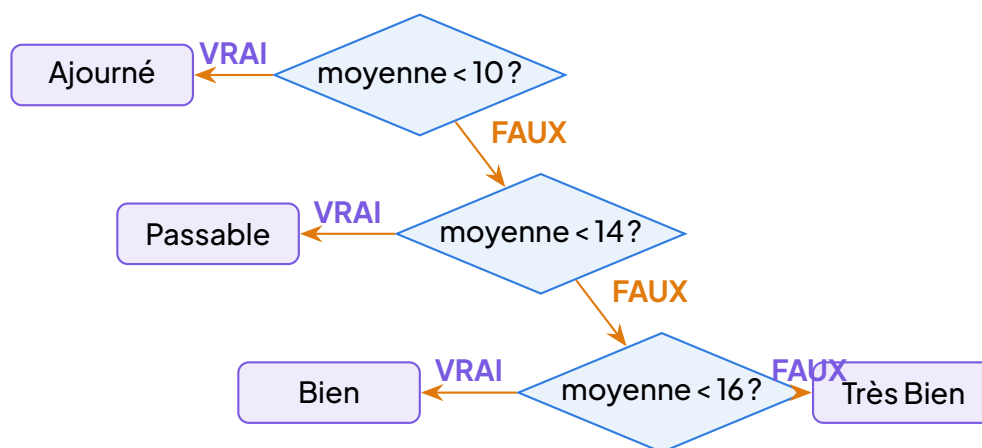
```

On teste les conditions dans l'ordre : dès qu'une condition est vraie, son bloc est exécuté et tous les autres sont ignorés.

**EXEMPLE** Mentions au BEPC

```
Algorithme Mention
Variable moyenne : Reel
Debut
  Ecrire("Entrez la moyenne : ")
  Lire(moyenne)
  Si moyenne < 10 Alors
    Ecrire("Ajourne")
  Sinon
    Si moyenne < 14 Alors
      Ecrire("Mention Passable")
    Sinon
      Si moyenne < 16 Alors
        Ecrire("Mention Bien")
      Sinon
        Ecrire("Mention Tres Bien")
      FinSi
    FinSi
  FinSi
Fin
```

Remarque l'astuce : grâce à l'ordre des tests, la condition « moyenne < 14 » n'est examinée que si l'on sait déjà que moyenne ≥ 10. Inutile donc d'écrire « moyenne ≥ 10 ET moyenne < 14 », même si ce serait correct.



Arbre de décision à trois niveaux pour les mentions : chaque losange est un test, et la branche « FAUX » mène au test suivant.

## 2.5 Représentation par algorithme : symboles normalisés

Le programme officiel demande de savoir représenter un algorithme par un algorithme. Contrairement à l'arbre de décision qui ne montre que la logique des tests, l'algorithme utilise des symboles normalisés (norme ISO 5807) pour décrire tout l'algorithme, du début à la fin.

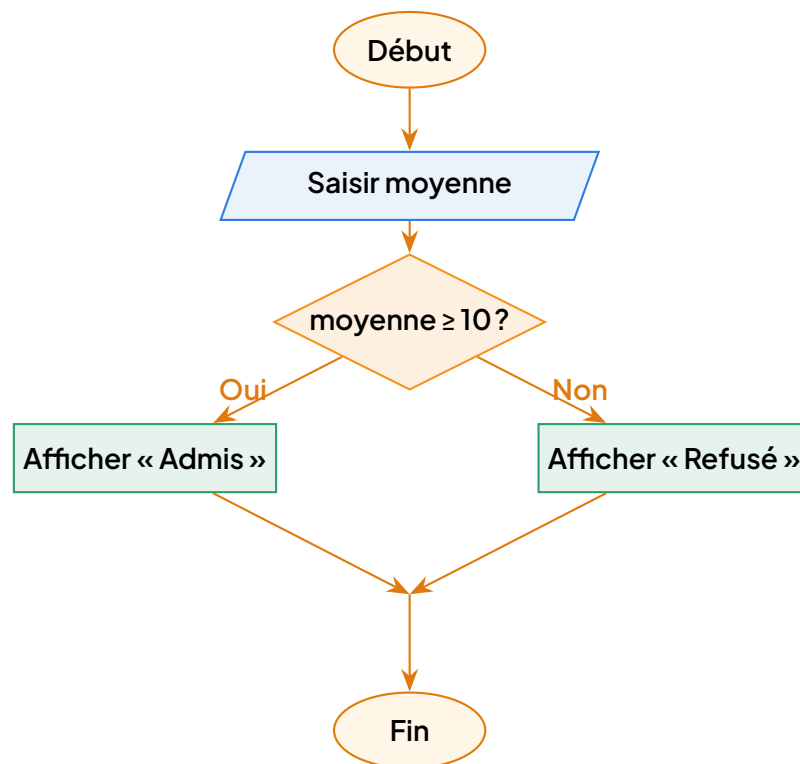
**DÉFINITION** Symboles de base de l'algorithme

- Ovale : Début et Fin de l'algorithme.
- Parallélogramme : Entrée (Saisir/Lire) et Sortie (Afficher/Écrire).
- Rectangle : Traitement / Affectation (ex : `prix ← montant * 0,95`).
- Losange : Test / Condition (Structure alternative). Il a deux sorties : « Oui » (VRAI) et « Non » (FAUX).
- Flèches : Sens d'exécution (généralement de haut en bas).

**MÉTHODE** Dessiner l'algorithme d'une alternative complète

1. Début (Ovale).
2. Saisir les données d'entrée (Parallélogramme).
3. Tester la condition (Losange). Écrire la condition à l'intérieur.
4. Dessiner la branche Oui (VRAI) : y placer les instructions du **Alors** (Rectangles ou Parallélogrammes).
5. Dessiner la branche Non (FAUX) : y placer les instructions du **Sinon**.
6. Rejoindre les deux branches après le **FinSi**.
7. Fin (Ovale).

**EXEMPLE** Algorithme complet : Algorithme « Resultat » (Admis/Refusé)



Algorithme normalisé de l'algorithme « Resultat ». Notez la jonction des deux branches après le test.

**ATTENTION** Deux erreurs très fréquentes au BEPC

- Oublier le `FinSi` : chaque `Si` doit être fermé par un `FinSi`. Avec trois alternatives imbriquées, il faut trois `FinSi`. Comptez-les toujours à la fin de l'écriture !
- Se tromper d'opérateur (piège des bornes) : si l'énoncé dit « admis à partir de 10 », un élève qui a exactement 10 doit être admis. Il faut donc écrire `Si moyenne >= 10` et non `Si moyenne > 10`, qui exclurait la note 10. De même, « dépasse 10000 » signifie `> 10000`, tandis que « à partir de 10000 » signifie `>= 10000`. Lis l'énoncé avec la plus grande attention.
- Mauvaise indentation : dans les alternatives imbriquées, décale le code vers la droite à chaque niveau de `Si`. Cela rend la lecture visuelle des `FinSi` correspondants beaucoup plus facile.

**LESAVAIS-TU ? Les alternatives dans ton téléphone**

Les applications de transfert d'argent comme Orange Money ou MTN Mobile Money, très utilisées au Cameroun, exécutent des alternatives à chaque transaction. Quand tu envoies de l'argent, le système teste d'abord : «  $\text{solde} \geq \text{montant} + \text{frais} ?$  ». Si c'est VRAI, le transfert est effectué ; Sinon, tu reçois le message « Solde insuffisant ». Des millions de décisions de ce type sont ainsi prises automatiquement chaque jour !

**EXERCICE 2 À toi de jouer**

1. Écris un algorithme qui lit l'âge d'une personne et affiche « Mineur » si l'âge est inférieur à 18 ans, et « Majeur » sinon. Dessine l'algorithme correspondant.
2. Un cybercafé facture l'heure de connexion à 300 FCFA, mais offre une heure gratuite à tout client qui achète plus de 5 heures. Écris l'algorithme qui lit le nombre d'heures achetées et affiche le montant à payer.
3. Reprends l'algorithme des mentions (section « Alternatives imbriquées ») et fais sa trace (sous forme de tableau) pour les moyennes 8, 12, 15 et 17.
4. Défi : Écris un algorithme qui lit trois nombres  $a, b, c$  et affiche le plus grand des trois. (Indication : utilise des alternatives imbriquées ou une variable  $\text{max}$ ).

**CORRIGÉ Exercice (éléments de correction)**

1. **Algorithme** : Si  $\text{age} < 18$  Alors Ecrire("Mineur") Sinon Ecrire("Majeur") FinSi.  
**Algorithme** : Début → Saisir  $\text{age}$  → Losange ( $\text{age} < 18 ?$ ) → Oui : Afficher « Mineur » / Non : Afficher « Majeur » → Jonction → Fin.
2. **Algorithme Cyber**  
Variable heures, payer : Entier  
Début  
Ecrire("Heures achetées : ")  
Lire(heures)  
Si heures > 5 Alors  
payer ← (heures - 1) \* 300  
Sinon  
payer ← heures \* 300  
FinSi  
Ecrire("Montant : ", payer)  
Fin
3. Traces :

- 8 : Test1 ( $8 < 10$ ) VRAI → « Ajourné ».
- 12 : Test1 FAUX → Test2 ( $12 < 14$ ) VRAI → « Mention Passable ».
- 15 : Test1 FAUX → Test2 FAUX → Test3 ( $15 < 16$ ) VRAI → « Mention Bien ».
- 17 : Test1 FAUX → Test2 FAUX → Test3 FAUX → « Mention Tres Bien ».

4. **Algorithme Max3** : Si  $a \geq b$  Alors  
 Si  $a \geq c$  Alors  $\max \leftarrow a$  Sinon  $\max \leftarrow c$  FinSi  
 Sinon  
 Si  $b \geq c$  Alors  $\max \leftarrow b$  Sinon  $\max \leftarrow c$  FinSi  
 FinSi  
 Ecrire( $\max$ ).

#### À RETENIR L'essentiel de la section

- Une **condition** est une expression qui vaut VRAI ou FAUX ; elle s'écrit avec les opérateurs  $=, \neq, <, >, \leq, \geq$  et peut combiner plusieurs tests avec ET et OU.
- L'**alternative simple** (Si ...Alors ...FinSi) n'agit que si la condition est vraie.
- L'**alternative complète** (Si ...Alors ...Sinon ...FinSi) choisit entre deux actions : l'une des deux est toujours exécutée.
- Pour plus de deux cas, on imbrique les alternatives : chaque Si doit avoir son FinSi.
- L'**algorithme** utilise des symboles normalisés : Ovale (Début/Fin), Parallélogramme (Entrée/Sortie), Rectangle (Traitement), Losange (Test).
- Attention aux bornes : « à partir de 10 » se traduit par  $\geq 10$ , pas par  $> 10$ .

## 3 LES STRUCTURES ITÉRATIVES : LA BOUCLE « TANT QUE »

Dans la section précédente, nous avons découvert les structures alternatives : grâce à Si ... Alors ... Sinon ... FinSi, un algorithme peut choisir entre plusieurs chemins selon une condition. Mais un choix, même bien fait, ne s'exécute qu'une seule fois. Or, dans la vie courante, beaucoup de tâches consistent à répéter la même action : compter les élèves d'une classe un par un, ajouter de l'argent à une épargne chaque semaine, afficher les dix premiers nombres entiers... Écrire dix fois la même instruction serait fastidieux, et mille fois, impossible. C'est précisément le problème que résolvent les **structures itératives**, aussi appelées **boucles**. Dans cette section, nous étudions la plus intuitive d'entre elles : la boucle TantQue.

### 3.1 Notion de boucle : répéter tant qu'une condition est vraie

#### EXPÉRIENCE Une situation pour comprendre : l'épargne d'Amina

Amina, élève de 3ème, veut acheter un livre de préparation au BEPC qui coûte 50000 FCFA. Chaque semaine, elle dépose 5000 FCFA dans sa cagnotte. Observe sa démarche :

1. Elle regarde le contenu de sa cagnotte : a-t-elle atteint 50000 FCFA?
2. Si non, elle ajoute 5000 FCFA et recommence à l'étape 1.
3. Si oui, elle s'arrête : elle peut acheter son livre.

Observation : Amina répète la même action (ajouter 5000 FCFA) tant que le total est inférieur à 50000 FCFA. Elle ne sait pas à l'avance combien de fois elle répétera l'action en la comptant mentalement : elle s'arrête quand la condition devient fausse. C'est exactement le fonctionnement de la boucle `TantQue`.

#### DÉFINITION Structure itérative et boucle `TantQue`

Une **structure itérative** (ou **boucle**) est une structure de contrôle qui permet d'exécuter plusieurs fois un même bloc d'instructions.

La boucle `TantQue` exécute un bloc d'instructions tant qu'une condition est vraie :

- on teste d'abord la condition ;
- si la condition est vraie, on exécute le bloc d'instructions (appelé corps de la boucle), puis on revient tester la condition ;
- si la condition est fausse, on sort de la boucle et on continue l'algorithme après le `FinTantQue`.

Chaque exécution du corps de la boucle s'appelle une **itération**.

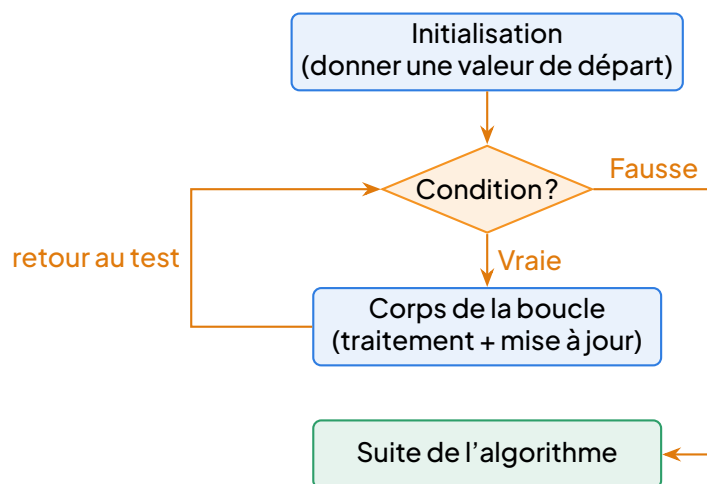
#### PROPRIÉTÉ Syntaxe de la boucle `TantQue`

En pseudo-code, la boucle `TantQue` s'écrit :

```
TantQue condition Faire  
    instructions  
FinTantQue
```

- `condition` est une expression logique (vraie ou fausse), par exemple `total < 50000` ;
- le test a lieu avant chaque itération, y compris la première : si la condition est fausse dès le départ, le corps de la boucle n'est jamais exécuté ;

- les instructions du corps sont indentées (décalées vers la droite) pour bien montrer qu'elles appartiennent à la boucle.



Algorithme de la boucle TantQue : on teste la condition; si elle est vraie, on exécute le corps puis on revient au test; si elle est fausse, on sort de la boucle.

### 3.2 Les trois ingrédients d'une boucle correcte

Pour qu'une boucle TantQue fonctionne bien, trois éléments doivent toujours être présents. Leur oubli est la cause de presque toutes les erreurs.

#### MÉTHODE Construire une boucle TantQue correcte

1. Initialiser : avant la boucle, donner une valeur de départ à la variable utilisée dans la condition (exemple : `total ← 0`). Sans initialisation, la condition porte sur une valeur inconnue.
2. Tester : écrire la condition qui doit être vraie pour continuer (exemple : `TantQue total < 50000`).
3. Traiter et mettre à jour : dans le corps de la boucle, effectuer le travail voulu et modifier la variable de la condition (exemple : `total ← total + 5000`), afin que la condition puisse devenir fausse.

On retient le réflexe : **initialiser**, **tester**, **traiter**, **mettre à jour**.

**ATTENTION** Erreur fréquente : la boucle infinie

Si l'on oublie la mise à jour de la variable dans le corps de la boucle, la condition reste vraie pour toujours : la boucle ne s'arrête jamais. On dit qu'on a créé une **boucle infinie**. L'ordinateur exécute alors les mêmes instructions sans fin, et le programme « plante ».

```
total <- 0
TantQue total < 50000 Faire
    Afficher("Pas encore assez...")
    // ERREUR : total n'est jamais modifié !
FinTantQue
```

Ici, `total` vaut 0 pour l'éternité : la condition `total < 50000` reste vraie indéfiniment. Vérifie toujours que quelque chose change dans la boucle.

**PROPRIÉTÉ** Arrêt d'une boucle TantQue (admis)

Une boucle `TantQue` s'arrête si et seulement si sa condition finit par devenir fausse. Pour cela, il faut que les instructions du corps modifient au moins une variable de la condition, dans le sens qui rapproche de l'arrêt. Avant d'utiliser une boucle, on doit toujours se poser la question : « qu'est-ce qui garantit que la condition deviendra fausse ? ». Sur l'exemple de l'épargne, `total` augmente de 5000 FCFA à chaque tour : il dépassera forcément 50000 FCFA, donc la boucle s'arrête.

### 3.3 Exemple 1 : compter de 1 à 10

**EXEMPLE** Afficher les nombres de 1 à 10

On veut afficher les nombres 1, 2, 3, ..., 10. Sans boucle, il faudrait dix instructions `Afficher`. Avec une boucle `TantQue`, trois lignes suffisent :

```
Algorithme Compter
Variable
    i : Entier
Debut
    i <- 1           // Initialisation : on commence à 1
    TantQue i <= 10 Faire // Condition : on continue tant que i ne dépasse pas 10
        Afficher(i)   // Traitement : afficher le nombre courant
        i <- i + 1     // Mise à jour : passer au nombre suivant
    FinTantQue
Fin
```

Déroulement : `i` vaut 1, la condition `i <= 10` est vraie, on affiche 1 puis `i` devient 2 ; on re-teste, on affiche 2, etc. Quand `i` vaut 11, la condition devient fausse : on sort de la boucle. La boucle a effectué exactement 10 itérations.

### 3.4 Exemple 2 : l'épargne hebdomadaire d'Amina

Reprenons la situation de départ et traduisons-la en algorithme complet.

#### EXEMPLE Combien de semaines pour atteindre 50 000 FCFA ?

```

Algorithme Epargne
Variable
    total, semaine : Entier
Debut
    total <- 0           // Initialisation : cagnotte vide
    semaine <- 0         // Aucune semaine écoulée
    TantQue total < 50000 Faire // On continue tant que l'objectif n'est pas atteint
        total <- total + 5000 // Mise à jour : dépôt hebdomadaire
        semaine <- semaine + 1 // Une semaine de plus
    FinTantQue
    Afficher("Objectif atteint en ", semaine, " semaines.")
Fin
  
```

Pour vérifier qu'un algorithme est correct, on réalise une **trace d'exécution** : on joue soi-même le rôle de l'ordinateur et on note, dans un tableau, la valeur de chaque variable à la fin de chaque itération.

| Semaine  | Montant ajouté (FCFA) | Total (FCFA) | Condition <code>total &lt; 50000</code> |
|----------|-----------------------|--------------|-----------------------------------------|
| (départ) | —                     | 0            | Vraie : on entre dans la boucle         |
| 1        | 5 000                 | 5 000        | Vraie                                   |
| 2        | 5 000                 | 10 000       | Vraie                                   |
| 3        | 5 000                 | 15 000       | Vraie                                   |
| 4        | 5 000                 | 20 000       | Vraie                                   |
| 5        | 5 000                 | 25 000       | Vraie                                   |
| 6        | 5 000                 | 30 000       | Vraie                                   |
| 7        | 5 000                 | 35 000       | Vraie                                   |
| 8        | 5 000                 | 40 000       | Vraie                                   |
| 9        | 5 000                 | 45 000       | Vraie                                   |
| 10       | 5 000                 | 50 000       | Fausse : on sort de la boucle           |

Au bout de la 10<sup>e</sup> semaine, `total` vaut 50 000 FCFA : la condition `total < 50000` devient fausse, la boucle s'arrête et l'algorithme affiche : « Objectif atteint en 10 semaines. » On retrouve bien le calcul  $50\,000 \div 5\,000 = 10$ .

#### EXERCICE RÉSOLU 3 Lecture de SMS : compter les messages non lus

Énoncé. Le téléphone de la maman de Rodrigue affiche ses SMS un par un. Écris un algorithme qui, partant de 25 messages non lus, en lit 5 par jour et affiche au bout de combien

de jours il ne restera plus de message non lu. Donne la trace d'exécution.

**SOLUTION**

Solution. On initialise `reste` à 25 et `jour` à 0. Tant qu'il reste des messages, on en lit 5 et on compte un jour.

```
Algorithme LectureSMS
Variable
    reste, jour : Entier
Debut
    reste <- 25
    jour <- 0
    TantQue reste > 0 Faire
        reste <- reste - 5    // Mise à jour : 5 messages lus
        jour <- jour + 1
    FinTantQue
    Afficher("Tous les messages sont lus en ", jour, " jours.")
Fin
```

Trace d'exécution : `reste` prend les valeurs 20, 15, 10, 5 puis 0, et `jour` prend 1, 2, 3, 4, 5. Quand `reste` vaut 0, la condition `reste > 0` est fausse : on sort. L'algorithme affiche « Tous les messages sont lus en 5 jours. » La mise à jour (`reste` diminue de 5) garantit l'arrêt : pas de boucle infinie.

**À RETENIR** L'essentiel sur la boucle TantQue

- Une boucle permet de répéter des instructions ; la boucle `TantQue` répète tant qu'une condition est vraie, en testant la condition avant chaque itération.
- **Syntaxe** : `TantQue condition Faire ... FinTantQue`.
- Une boucle correcte exige toujours : une initialisation avant la boucle, un traitement et une mise à jour dans la boucle.
- Sans mise à jour, la condition reste vraie : c'est la boucle infinie, l'erreur à éviter absolument.
- La trace d'exécution (tableau des valeurs des variables) est l'outil pour vérifier qu'une boucle fait bien ce qu'on attend.

**EXERCICE 3** À toi de jouer

1. Écris un algorithme qui affiche les nombres pairs de 2 à 20 à l'aide d'une boucle `TantQue`.
2. Un commerçant de Mokolo dépose 10000 FCFA par jour dans sa caisse. Écris un algo-

rithme qui calcule en combien de jours il atteindra 150000 FCFA, puis donne sa trace d'exécution.

3. L'algorithme suivant contient une erreur. Trouve-la et corrige-la.

```
i <- 1
TantQue i <= 5 Faire
    Afficher(i)
FinTantQue
```

#### **CORRIGÉ** Exercice : À toi de jouer

1. On initialise *i* à 2 et on avance de 2 en 2 :

```
i <- 2
TantQue i <= 20 Faire
    Afficher(i)
    i <- i + 2
FinTantQue
```

2. *caisse* <- 0, *jour* <- 0; **tant que** *caisse* < 150000, on ajoute 10000 FCFA et un jour. La trace donne 10 000, 20 000, ..., 150 000 : l'objectif est atteint en 15 jours (150 000 ÷ 10 000 = 15).
3. Il manque la mise à jour de *i* dans la boucle : la condition *i* <= 5 reste vraie pour toujours, c'est une boucle infinie. Correction : ajouter *i* <- *i* + 1 après *Afficher(i)*.

#### **LESAVAIS-TU ?** Et dans un vrai programme ?

La boucle **TantQue** existe dans tous les langages de programmation : elle s'écrit **while** en anglais (mot qui signifie « tant que »). C'est grâce à des boucles qu'un distributeur automatique de la SGC ou d'Afriland recompte ton solde après chaque opération, et que le logiciel de la cantine scolaire parcourt la liste de tous les élèves inscrits, un par un, sans jamais en oublier.

## 4

## LES STRUCTURES ITÉRATIVES : LA BOUCLE « POUR »

Dans la section précédente, nous avons découvert la boucle **TantQue** : elle répète des instructions tant qu'une condition reste vraie, et c'est le programmeur qui doit gérer lui-même l'initialisation et la progression de la variable de contrôle. Cette boucle est très souple, mais elle demande de la vigilance (risque de boucle infinie si l'on oublie de faire progresser la condition).

Or, dans de très nombreuses situations de la vie courante, on sait **à l'avance combien de fois** on veut répéter une action : afficher les notes des 25 élèves d'une classe de 3ème, calculer la paie des 12 mois de l'année, afficher la table de multiplication de 7 (de  $1 \times 7$  jusqu'à  $10 \times 7$ ). Pour ces cas, l'algorithmique nous offre une boucle plus simple et plus sûre : la boucle **Pour**.

#### 4.1 Quand utilise-t-on la boucle « Pour » ?

Imaginons la cantine du collège : la cuisinière doit distribuer un repas à chacun des 100 élèves inscrits. Elle ne se demande pas « est-ce que je continue ? » à chaque assiette : elle sait qu'elle fera exactement 100 distributions, ni plus, ni moins. Le nombre de répétitions est **connu à l'avance**.

##### DÉFINITION La boucle Pour

La boucle **Pour** (dite aussi boucle « à compteur ») est une structure itérative qui répète un bloc d'instructions un nombre de fois connu à l'avance. Elle utilise une variable appelée **compteur**, qui prend automatiquement, tour après tour, toutes les valeurs entières comprises entre une valeur de départ et une valeur d'arrivée.

##### DÉFINITION Le compteur

Le **compteur** est la variable qui pilote la boucle **Pour**. Il est initialisé automatiquement à la valeur de départ, puis augmenté automatiquement d'une quantité fixe appelée le **pas** (par défaut, le pas vaut 1) à la fin de chaque tour. La boucle s'arrête lorsque le compteur dépasse la valeur d'arrivée.

La grande différence avec le **TantQue** est donc là : avec le **Pour**, le programmeur n'a **ni à initialiser**, **ni à incrémenter** le compteur lui-même. La boucle s'en charge. C'est ce qui la rend plus sûre : on ne peut pas oublier l'incrémentation, donc pas de boucle infinie par oubli.

#### 4.2 Syntaxe de la boucle « Pour »

##### PROPRIÉTÉ Syntaxe générale de la boucle Pour

En pseudo-code, la boucle Pour s'écrit :

```
Pour compteur de valeur_debut a valeur_fin Faire
  instruction 1
  instruction 2
  ...
FinPour
```

- **compteur** : une variable de type entier ;
- **valeur\_debut** et **valeur\_fin** : deux expressions entières, avec  $\text{valeur\_debut} \leq \text{valeur\_fin}$  ;

- à chaque tour, le compteur augmente automatiquement de 1 (c'est le **pas** par défaut);
- la boucle s'arrête dès que le compteur dépasse `valeur_fin`.

**Note :** on peut préciser un pas différent de 1 avec la clause optionnelle `Pas valeur_pas` (ex. Pour *i* de 2 à 20 Pas 2 Faire).

**Le pas différent de 1 (complément).** Dans certains langages et dans certaines écritures de pseudo-code, on peut préciser un **pas** différent de 1 : par exemple Pour *i* de 2 à 20 Pas 2 fait prendre à *i* les valeurs 2, 4, 6, ..., 20 (les nombres pairs). On peut même utiliser un pas négatif pour compter à rebours : Pour *i* de 10 à 1 Pas -1 produit 10, 9, 8, ..., 1. Au niveau de la classe de 3ème, on retient surtout le cas du pas égal à 1, mais il est utile de savoir que cette possibilité existe.

### 4.3 Premier exemple : la somme des entiers de 1 à 100

**Problème.** On veut calculer  $S = 1 + 2 + 3 + \dots + 100$ . Le nombre d'additions à effectuer est parfaitement connu : 100. C'est donc le cas d'école de la boucle Pour.

**Idée.** On utilise une variable `somme`, initialisée à 0, qui va « accumuler » les entiers un par un : au tour où le compteur *i* vaut 1, on ajoute 1; au tour où il vaut 2, on ajoute 2; et ainsi de suite jusqu'à 100.

#### EXEMPLE Algorithme : somme des entiers de 1 à 100

```
Algorithme Somme1a100
Variables
  i, somme : Entier
Debut
  somme <- 0           // l'accumulateur part de zero
  Pour i de 1 à 100 Faire
    somme <- somme + i // on ajoute la valeur du compteur
  FinPour
  Ecrire("La somme vaut : ", somme)
Fin
```

Déroulons mentalement les premiers tours : avant la boucle, `somme` vaut 0. Au premier tour, *i* vaut 1 et `somme` devient  $0 + 1 = 1$ . Au deuxième tour, *i* vaut 2 et `somme` devient  $1 + 2 = 3$ . Au troisième tour, `somme` devient  $3 + 3 = 6$ . On voit que la variable `somme` contient à chaque instant la somme de tous les entiers déjà rencontrés : c'est pourquoi on l'appelle un **accumulateur**.

#### EXERCICE RÉSOLU 4 Trace de la somme $1 + 2 + \dots + 10$ et vérification

**Énoncé :** pour bien comprendre le fonctionnement de l'accumulateur, on exécute « à la main » l'algorithme précédent en le limitant à 10 (au lieu de 100). Compléter le tableau de trace donnant, à chaque tour, la valeur de *i* et celle de `somme` après l'instruction `somme <-`

somme + i. Quelle valeur finale obtient-on ? Vérifier avec la formule  $\frac{n(n+1)}{2}$ .

**SOLUTION**

Solution détaillée : on part de `somme = 0` et on ajoute successivement 1, 2, 3, ..., 10 :

| Tour (valeur de i)  | 1 | 2 | 3 | 4  | 5  | 6  | 7  | 8  | 9  | 10 |
|---------------------|---|---|---|----|----|----|----|----|----|----|
| somme après le tour | 1 | 3 | 6 | 10 | 15 | 21 | 28 | 36 | 45 | 55 |

La valeur finale affichée est donc `somme = 55`. Vérification par la formule de la somme des  $n$  premiers entiers :

$$S = \frac{n(n+1)}{2} = \frac{10 \times 11}{2} = \frac{110}{2} = 55.$$

Les deux résultats concordent : l'algorithme est correct. Pour  $n = 100$ , la même formule donne  $S = \frac{100 \times 101}{2} = \frac{10100}{2} = 5050$ , ce qui confirme le résultat affiché par l'algorithme.

**LESAVAIS-TU ? Le jeune Gauss et la somme de 1 à 100**

Carl Friedrich Gauss (1777–1855), l'un des plus grands mathématiciens de l'histoire, aurait ébloui son instituteur dès l'âge de 8 ans. Pour occuper la classe, le maître avait demandé de calculer  $1 + 2 + 3 + \dots + 100$ , pensant que cela prendrait longtemps. Le jeune Gauss aurait répondu presque instantanément : 5050 ! Son astuce ? Regrouper les nombres par paires :  $1 + 100 = 101$ ,  $2 + 99 = 101$ ,  $3 + 98 = 101$ , etc. Il y a 50 paires, chacune valant 101, donc  $50 \times 101 = 5050$ . C'est exactement la formule  $\frac{n(n+1)}{2}$ . Ton ordinateur, lui, fait les 100 additions une par une avec la boucle `Pour...` mais il les fait en une fraction de seconde !

**4.4 Deuxième exemple : la table de multiplication de 7**

**Problème.** Un élève de 3ème veut afficher la table de multiplication de 7, de  $1 \times 7$  jusqu'à  $10 \times 7$ . Le nombre de lignes à afficher est connu : 10. Encore un cas parfait pour la boucle `Pour`.

**EXEMPLE Algorithme : table de multiplication de 7**

```

Algorithme TableDe7
Variables
  i, produit : Entier
Debut
  Pour i de 1 a 10 Faire
    produit <- 7 * i
    Ecrire("7 x ", i, " = ", produit)
  FinPour

```

Fin

À chaque tour, le compteur `i` vaut successivement 1, 2, 3, ..., 10, et l'instruction `Ecrire` affiche la ligne correspondante : « 7 x 1 = 7 », puis « 7 x 2 = 14 », ..., jusqu'à « 7 x 10 = 70 ». Remarque que le compteur n'est pas seulement un « numéro de tour » : on l'**utilise dans les calculs** du corps de la boucle. C'est très fréquent et très pratique.

#### 4.5 Équivalence entre « Pour » et « TantQue »

On peut se poser la question : la boucle `Pour` est-elle vraiment une nouveauté, ou bien peut-on la « traduire » avec ce que l'on connaît déjà ? La réponse est importante :

##### PROPRIÉTÉ Équivalence des deux boucles

Toute boucle **Pour** peut être réécrite avec une boucle **TantQue**. Il suffit de faire « à la main » ce que le `Pour` fait automatiquement :

1. initialiser le compteur à la valeur de départ, avant la boucle ;
2. écrire la condition `compteur <= valeur_fin` dans le `TantQue` ;
3. incrémenter le compteur (`compteur <- compteur + 1`) à la fin du corps de la boucle.

La réciproque est fautive : un `TantQue` dont on ne connaît pas le nombre de tours à l'avance (par exemple « tant que le mot de passe saisi est faux ») ne peut pas toujours s'écrire avec un `Pour`.

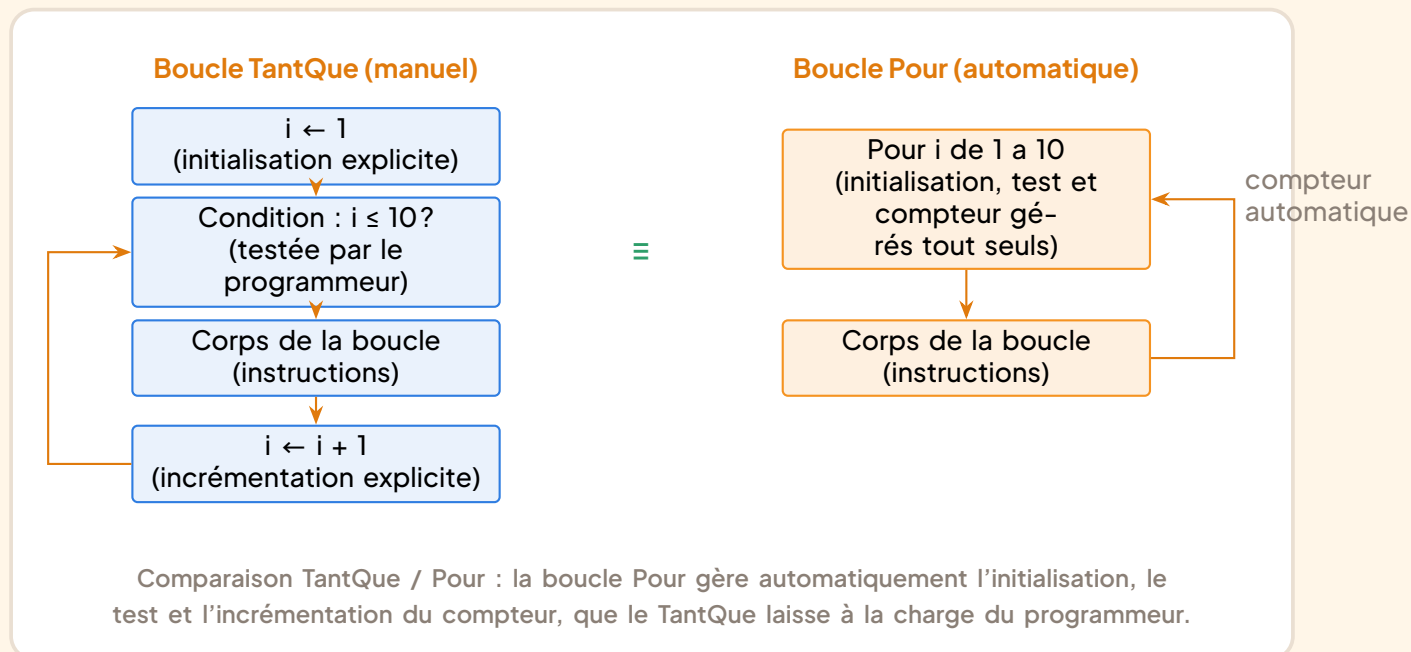
##### EXEMPLE Réécriture guidée : la table de 7 avec un TantQue

Reprenons l'algorithme de la table de 7 et appliquons la méthode de traduction, étape par étape :

```
Algorithme TableDe7Bis
Variables
  i, produit : Entier
Debut
  i <- 1                // etape 1 : initialisation du compteur
  TantQue i <= 10 Faire // etape 2 : condition de continuation
    produit <- 7 * i
    Ecrire("7 x ", i, " = ", produit)
    i <- i + 1          // etape 3 : incrementation (a ne pas oublier !)
  FinTantQue
Fin
```

Ce programme affiche exactement les mêmes 10 lignes que la version avec `Pour`. On retrouve, écrites explicitement, les trois opérations que la boucle `Pour` réalisait automatiquement.

Le schéma suivant résume visuellement cette correspondance :



#### 4.6 Comment choisir entre « Pour » et « TantQue » ?

##### MÉTHODE Choisir la bonne boucle

Face à un problème qui demande de répéter des instructions, pose-toi une seule question : « Est-ce que je connais le nombre de répétitions avant de commencer ? »

1. Oui, le nombre de tours est connu (les 25 élèves de la classe, les 12 mois de l'année, les entiers de 1 à 100)  $\Rightarrow$  utilise la boucle **Pour**. Elle est plus courte à écrire et plus sûre (pas de risque d'oublier l'incréméntation).
2. Non, l'arrêt dépend d'une condition qu'on ne peut pas prévoir (tant que le mot de passe est faux, tant que la saisie est invalide, tant que le stock n'est pas épuisé)  $\Rightarrow$  utilise la boucle **TantQue**.
3. En cas de doute, le **TantQue** convient toujours (puisque tout **Pour** peut s'écrire avec un **TantQue**), mais le **Pour** rend l'algorithme plus lisible quand le nombre de tours est fixe.

| Critère                  | Boucle Pour                              | Boucle TantQue                                    |
|--------------------------|------------------------------------------|---------------------------------------------------|
| Nombre de répétitions    | Connu à l'avance                         | Inconnu : dépend d'une condition                  |
| Compteur                 | Initialisé et incrémenté automatiquement | Géré manuellement par le programmeur              |
| Risque de boucle infinie | Quasi nul                                | Réel si l'on oublie de faire évoluer la condition |
| Exemple typique          | Somme de 1 à 100, table de 7             | Saisie d'un mot de passe correct                  |

**ATTENTION** Ne jamais modifier le compteur à l'intérieur d'une boucle Pour

Une erreur fréquente consiste à écrire, dans le corps d'une boucle `Pour`, une instruction qui modifie le compteur, par exemple :

```
Pour i de 1 à 10 Faire
  i <- i + 1      // INTERDIT : le compteur est déjà géré par le Pour !
  Ecrire(i)
FinPour
```

Le compteur d'une boucle `Pour` est géré automatiquement : le modifier à la main rend le comportement imprévisible (tours sautés, boucle qui ne s'arrête pas comme prévu, résultats faux). Règle d'or : dans le corps d'un `Pour`, on lit la valeur du compteur (pour calculer ou afficher), mais on ne lui affecte jamais de nouvelle valeur. Si tu sens le besoin de contrôler toi-même la progression, c'est le signe qu'il faut utiliser un `TantQue`.

**EXERCICE 4** À toi de jouer

1. Écris un algorithme qui calcule et affiche la somme des entiers pairs de 2 à 50 (utilise un compteur qui progresse de 2 en 2, ou un calcul adapté).
2. Le directeur d'un cybercafé veut afficher, pour chacun des 15 postes de la salle, le message « Poste n° X : contrôle effectué ». Rédige l'algorithme avec la boucle la plus adaptée et justifie ton choix.
3. Réécris l'algorithme de la question 2 avec une boucle `TantQue`, en appliquant la méthode de traduction vue dans cette section.

**CORRIGÉ** Corrigé de l'exercice

1. On initialise `somme <- 0`, puis `Pour i de 2 à 50 Pas 2 Faire somme <- somme + i`  
`FinPour`. Le résultat attendu est  $2 + 4 + \dots + 50 = 650$ .

2. Le nombre de passages est connu (15 postes) : on choisit donc la boucle **Pour**. Algorithme : Pour  $i$  de 1 à 15 Faire Ecrire("Poste n° ",  $i$ , " : contrôle effectué") FinPour.
3. On initialise  $i \leftarrow 1$ , la condition est TantQue  $i \leq 15$ , et on n'oublie pas  $i \leftarrow i + 1$  à la fin du corps de la boucle.

#### À RETENIR L'essentiel de la section

- La boucle **Pour** s'utilise lorsque le nombre de répétitions est connu à l'avance ; sa syntaxe est : Pour compteur de début à fin Faire ... FinPour.
- Le **compteur** est initialisé et augmenté automatiquement (pas de 1 par défaut) ; on ne le modifie jamais dans le corps de la boucle.
- Toute boucle **Pour** peut se réécrire avec un **TantQue** (initialisation avant, condition, incrémentation à la fin), mais l'inverse n'est pas toujours possible.
- Pour accumuler un résultat tour après tour (somme, total), on utilise un **accumulateur** initialisé à 0 : c'est ainsi qu'on obtient  $1 + 2 + \dots + 100 = 5050$ , résultat confirmé par la formule  $\frac{n(n+1)}{2}$ .

## 5 L'ALGORIGRAMME : REPRÉSENTATION GRAPHIQUE

Dans les sections précédentes, tu as appris à écrire des algorithmes en pseudo-code : des instructions écrites en français, avec des structures alternatives (Si... Alors... Sinon) et des structures itératives (Tant que, Pour). Ce texte est précis, mais il a un défaut : pour comprendre le déroulement d'un algorithme long, il faut lire chaque ligne, une par une. Un dessin, lui, se comprend d'un coup d'œil. C'est exactement le rôle de l'**algorithme** : transformer le texte d'un algorithme en un schéma clair, où l'on suit le cheminement du programme comme on suit un plan de métro.

### 5.1 Qu'est-ce qu'un algorithme ?

Imagine que tu dois expliquer à un camarade comment ton téléphone décide d'afficher « Admis » ou « Refusé » sur un bulletin. Tu peux lui réciter le pseudo-code... ou lui dessiner un schéma : « on entre la moyenne, on arrive à une question : la moyenne est-elle supérieure ou égale à 10 ? Si oui, on va par là ; si non, par ici. » Ce schéma, c'est un algorithme.

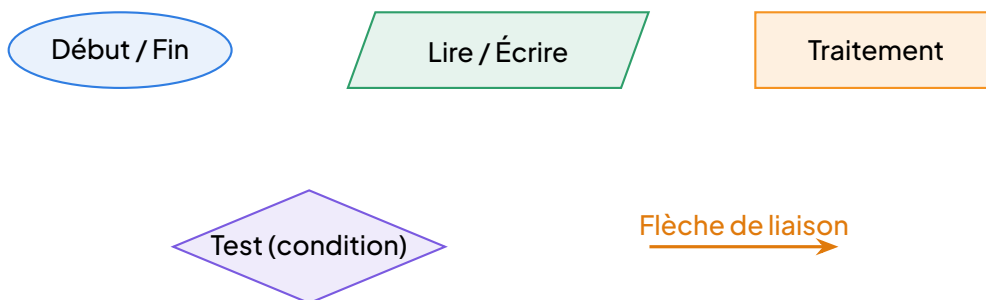
**DÉFINITION** Algorithme

Un **algorithme** (ou organigramme) est la représentation graphique d'un algorithme à l'aide de symboles normalisés reliés par des flèches. Chaque symbole correspond à un type d'opération (début, lecture, traitement, test, affichage), et les flèches indiquent l'ordre d'exécution des opérations.

L'algorithme et le pseudo-code décrivent donc le même algorithme, sous deux formes différentes : l'un est un dessin, l'autre est un texte. Savoir passer de l'un à l'autre est une compétence essentielle, que nous travaillerons à la fin de cette section.

## 5.2 Les symboles normalisés

Pour que tout le monde comprenne un algorithme, les symboles sont **normalisés** : ils ont une forme et un rôle précis, identiques partout dans le monde. En voici les quatre symboles de base à connaître, plus les flèches de liaison.

**À RETENIR** Rôle de chaque symbole

- L'**ovale** marque le Début ou la Fin de l'algorithme. Il y a toujours un seul Début et au moins une Fin.
- Le **parallélogramme** représente les entrées et sorties : Lire (l'utilisateur saisit une donnée) et Écrire (l'algorithme affiche un résultat).
- Le **rectangle** représente un traitement : un calcul ou une affectation (par exemple  $S \leftarrow S + i$ ).
- Le **losange** représente un test (une condition). Il possède exactement deux sorties : une branche Oui et une branche Non.
- Les **flèches** relient les symboles et indiquent le sens d'exécution.

**ATTENTION** Erreur fréquente : le test n'est pas un rectangle

L'erreur la plus classique consiste à dessiner un rectangle pour une condition comme « moyenne  $\geq 10$  ». C'est faux : un rectangle a une seule sortie, car un traitement ne pose aucune question. Un test est toujours un losange, avec exactement deux flèches sortantes étiquetées Oui et Non. De même, n'utilise jamais un losange pour un calcul : un calcul ne se pose pas de question, il s'exécute, donc c'est un rectangle.

### 5.3 Les flèches et le sens de lecture

Un algorithme se lit **de haut en bas** : le symbole Début est placé en haut, et l'exécution descend de flèche en flèche jusqu'au symbole Fin. Il n'y a qu'une seule exception à cette règle : les **boucles**, où une flèche remonte vers un symbole situé plus haut pour répéter des instructions. On parle alors de retour en arrière.

**PROPRIÉTÉ** Règles de construction des flèches

- Chaque symbole (sauf Fin) possède une flèche sortante ; chaque symbole (sauf Début) possède une flèche entrante.
- Un losange possède exactement deux flèches sortantes, étiquetées Oui et Non.
- Deux flèches ne se croisent pas si l'on peut l'éviter ; le schéma doit rester lisible.
- Le sens général de lecture est du haut vers le bas ; seule une boucle autorise une flèche qui remonte.

### 5.4 Représenter une alternative et une boucle

L'**alternative** (Si...Alors...Sinon) se traduit par un losange d'où partent deux branches : la branche Oui mène au traitement à faire quand la condition est vraie, la branche Non mène à l'autre traitement. Les deux branches **se rejoignent ensuite** pour continuer la suite de l'algorithme.

La **boucle** (Tant que ou Pour) se traduit par un losange dont la branche Oui effectue un traitement puis **revient en arrière** vers le losange : tant que la condition est vraie, on recommence. La branche Non quitte la boucle et poursuit l'algorithme.

**MÉTHODE** Construire un algorithme en quatre étapes

Pour dessiner l'algorithme d'un problème, procède toujours dans le même ordre :

1. Identifier les entrées : quelles données l'utilisateur doit-il saisir ? Ce seront des parallélogrammes Lire.

2. Identifier les traitements : quels calculs et affectations sont nécessaires? Ce seront des rectangles.
  3. Identifier les tests : où l'algorithme doit-il prendre une décision ou répéter des actions? Ce seront des losanges à deux branches.
  4. Identifier les sorties : quels résultats affiche-t-on? Ce seront des parallélogrammes.
- Écrire. Relie ensuite tous les symboles par des flèches, du Début vers la Fin.

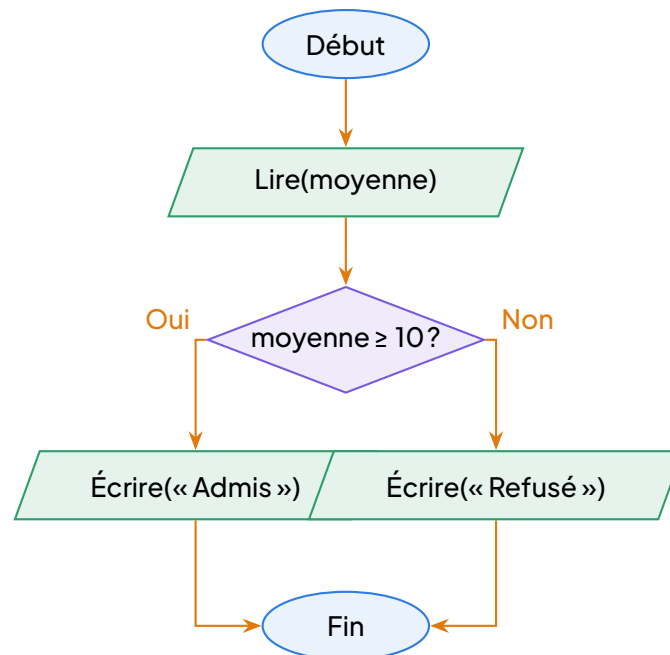
### 5.5 Exemple complet : l'algorithme Admis/Refusé

Reprenons notre algorithme classique : on saisit la moyenne d'un élève ; si elle est supérieure ou égale à 10, on affiche « Admis », sinon « Refusé ». Voici d'abord le pseudo-code, puis sa traduction graphique.

```
Algorithme AdmisRefuse
Variable moyenne : reel
Debut
    Lire(moyenne)
    Si moyenne >= 10 Alors
        Ecrire("Admis")
    Sinon
        Ecrire("Refuse")
    FinSi
Fin
```

#### EXEMPLE Conversion pseudo-code → algorithme : Admis/Refusé

Appliquons la méthode en quatre étapes. Entrée : la moyenne (parallélogramme Lire). Traitement : aucun calcul ici. Test : « moyenne  $\geq$  10 ? » (losange à deux branches). Sorties : « Admis » sur la branche Oui, « Refusé » sur la branche Non. Les deux branches se rejoignent avant le Fin. On obtient l'algorithme ci-dessous.



Algorithme de l'algorithme Admis/Refusé : le losange possède deux branches qui se rejoignent avant la fin.

Remarque comme le schéma se lit naturellement : on descend, on arrive à la question, on choisit un chemin, puis les deux chemins se rejoignent. C'est exactement le déroulement du pseudo-code, rendu visible.

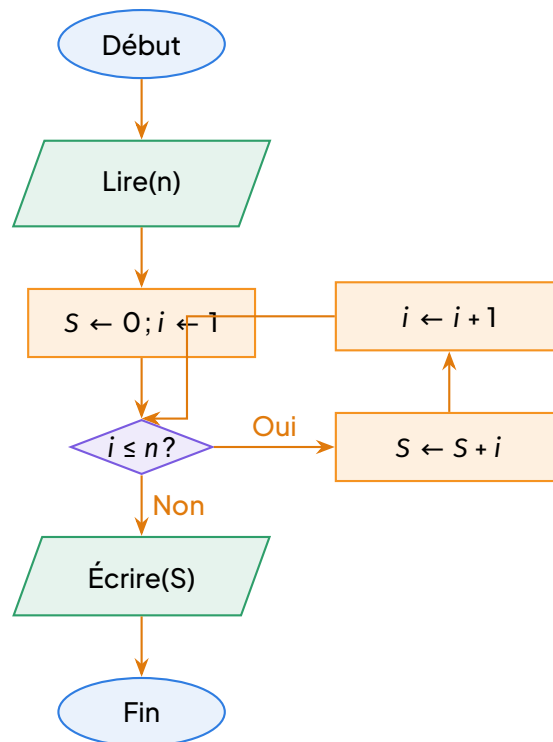
## 5.6 Exemple complet : la somme des entiers de 1 à n

Passons à un algorithme avec boucle : calculer  $S = 1 + 2 + 3 + \dots + n$ , où  $n$  est saisi par l'utilisateur. Le pseudo-code (avec une boucle Pour) est le suivant :

```

Algorithme Somme1aN
Variables n, i, S : entiers
Debut
  Lire(n)
  S ← 0
  Pour i allant de 1 à n Faire
    S ← S + i
  FinPour
  Ecrire("La somme est : ", S)
Fin
  
```

Pour le traduire en algorithme, on représente la boucle par un losange «  $i \leq n$  ? » : tant que la réponse est Oui, on exécute le traitement  $S \leftarrow S + i$ , on incrémente  $i$ , et une flèche **remonte** vers le losange. Quand la réponse devient Non, on sort de la boucle et on affiche le résultat.



Algorithme de la somme  $1 + 2 + \dots + n$  : la branche Oui du losange remonte vers le test, formant la boucle.

#### EXERCICE RÉSOLU 5 De l'algorithme au pseudo-code

On donne l'algorithme suivant, décrit symbole par symbole : Début; Lire(a); Lire(b); losange «  $a > b$  ? »; branche Oui : Écrire(a); branche Non : Écrire(b); Fin. Écris le pseudo-code correspondant et dis ce que fait cet algorithme.

#### SOLUTION

Solution. On suit les flèches de haut en bas : deux lectures, puis un test à deux branches, puis la fin. Le pseudo-code est :

```

Algorithme Maximum
Variables a, b : reels
Debut
  Lire(a)
  Lire(b)
  Si a > b Alors
    Écrire(a)
  Sinon
    Écrire(b)
  FinSi
Fin
  
```

Cet algorithme lit deux nombres et affiche le plus grand des deux. Vérification : si  $a = 7$  et  $b = 12$ , le test «  $7 > 12 ?$  » est faux, on suit la branche Non et on affiche 12 : c'est bien le maximum.

#### ATTENTION Pièges fréquents dans les boucles dessinées

- Oublier l'initialisation : le rectangle  $S \leftarrow 0 ; i \leftarrow 1$  doit être placé avant le losange, jamais dans la boucle, sinon  $S$  serait remis à zéro à chaque tour.
- Oublier l'incrémentation : sans le rectangle  $i \leftarrow i + 1$ , la condition  $i \leq n$  reste vraie pour toujours : la flèche remonte indéfiniment, c'est une boucle infinie.
- Inverser Oui et Non : la branche qui remonte doit correspondre au cas où l'on continue (condition vraie), la branche qui descend au cas où l'on s'arrête.

#### LESAVAIS-TU ? Les algorigrammes en entreprise

Dans les entreprises et les administrations — y compris au Cameroun, par exemple dans les services informatiques des banques ou de la CAMTEL — les développeurs dessinent souvent des algorigrammes avant d'écrire le moindre programme. Ce schéma sert de document de travail : il permet à toute l'équipe de discuter de la logique du futur logiciel, de repérer les oublis et les cas non prévus, puis de coder ensuite dans n'importe quel langage. Un bon dessin vaut parfois mille lignes de code !

#### EXERCICE 5 À toi de jouer

1. Dessine l'algorigramme de l'algorithme qui lit l'âge d'un élève et affiche « Mineur » si l'âge est inférieur à 18, « Majeur » sinon.
2. Dessine l'algorigramme de l'algorithme qui calcule et affiche la table de multiplication de 7 (de  $7 \times 1$  à  $7 \times 10$ ) à l'aide d'une boucle.
3. Pour chacun de tes algorigrammes, écris le pseudo-code correspondant.

#### CORRIGÉ Exercice

1. Symboles dans l'ordre : Début ; parallélogramme Lire(age) ; losange «  $\text{age} < 18 ?$  » ; branche Oui : parallélogramme Écrire(« Mineur ») ; branche Non : parallélogramme Écrire(« Majeur ») ; les deux branches se rejoignent ; Fin. Pseudo-code :

```
Algorithme Majorite
Variable age : entier
Debut
  Lire(age)
  Si age < 18 Alors
    Ecrire("Mineur")
  Sinon
    Ecrire("Majeur")
  FinSi
Fin
```

2. L'algorithme détaille le fonctionnement de la boucle Pour : Début ; rectangle  $i \leftarrow 1$  ; losange «  $i \leq 10 ?$  » ; branche Oui : rectangle  $p \leftarrow 7 \times i$ , puis parallélogramme Écrire(p), puis rectangle  $i \leftarrow i + 1$ , puis flèche de retour vers le losange ; branche Non : Fin. Pseudo-code :

```
Algorithme TableDe7
Variables i, p : entiers
Debut
  Pour i allant de 1 a 10 Faire
    p  $\leftarrow 7 \times i$ 
    Ecrire("7 x ", i, " = ", p)
  FinPour
Fin
```

3. Les pseudo-codes ci-dessus correspondent exactement aux algorigrammes : chaque parallélogramme donne un Lire ou un Écrire, chaque rectangle une affectation, chaque losange un Si ou une boucle.

#### À RETENIR L'essentiel de la section

- Un **algorithme** est la représentation graphique d'un algorithme : symboles normalisés reliés par des flèches, lu de haut en bas.
- Ovale = Début/Fin ; parallélogramme = Lire/Écrire ; rectangle = traitement ; losange = test avec exactement deux sorties Oui/Non.
- Une alternative se dessine par un losange dont les deux branches se rejoignent ; une boucle par un losange dont la branche Oui remonte vers le test.
- Algorithme et pseudo-code sont deux présentations du même algorithme : il faut savoir passer de l'une à l'autre.

## 6 RÉOLUTION DE PROBLÈMES CONCRETS ET JUSTIFICATION

Dans la section précédente, nous avons appris à représenter graphiquement un algorithme grâce à l'algorithme : rectangles pour les traitements, losanges pour les tests, flèches pour l'enchaînement des étapes. Nous savons donc maintenant écrire une séquence d'instructions, une alternative **Si...Alors...Sinon**, une boucle **Tant que** ou **Pour**, et dessiner tout cela. Mais un algorithme ne s'écrit jamais pour le plaisir : il sert à **résoudre un problème concret**. C'est exactement ce que l'on te demandera au BEPC : face à une situation de la vie courante (une facture, une moyenne de notes, un petit jeu), sauras-tu proposer une démarche complète et justifiée ? C'est l'objet de cette dernière section, qui est aussi la synthèse de toute la leçon.

### 6.1 La démarche complète de résolution d'un problème

Quand un élève reçoit un énoncé, sa première tentation est d'écrire immédiatement des instructions. C'est une erreur : on ne construit pas une maison sans plan. Un bon informaticien suit toujours une démarche en étapes, que l'on peut apprendre par cœur.

#### MÉTHODE

#### Résoudre un problème par un algorithme : les 6 étapes

1. Comprendre le problème. Lire l'énoncé plusieurs fois, le reformuler avec ses propres mots, repérer ce qui est demandé.
2. Identifier les entrées et les sorties. Les **entrées** sont les données fournies (exemple : la consommation en kWh) ; les **sorties** sont les résultats attendus (exemple : le montant à payer).
3. Choisir les structures de contrôle. Y a-t-il un cas particulier ? → structure alternative. Y a-t-il une action répétée ? → structure itérative (**Tant que** si l'on ne connaît pas le nombre de répétitions, **Pour** si on le connaît).
4. Écrire l'algorithme en pseudo-code, avec un nom, des variables déclarées, un **Début** et une **Fin**.
5. Tracer l'algorithme : le faire « tourner » à la main sur au moins deux jeux de valeurs tests, en notant l'évolution des variables dans un tableau.
6. Conclure et justifier : répondre à la question posée et expliquer pourquoi on a choisi telle structure plutôt qu'une autre.

#### À RETENIR

#### La trace d'exécution

La **trace** (ou table d'exécution) est un tableau où chaque colonne correspond à une variable et chaque ligne à une étape de l'exécution. Elle permet de vérifier qu'un algorithme

est correct avant de le faire exécuter par une machine.

## 6.2 Problème 1 : la facture d'électricité

**Situation.** Une compagnie d'électricité facture le kilowattheure (kWh) à 50 FCFA. Mais pour décourager le gaspillage, si la consommation mensuelle dépasse 110 kWh, le tarif passe à 75 FCFA par kWh pour toute la consommation. Écrire un algorithme qui lit la consommation et affiche le montant à payer.

**Étapes 1 et 2 — comprendre, identifier.** Entrée : la consommation  $C$  (en kWh). Sortie : le montant  $M$  (en FCFA). Il y a deux cas : c'est donc une **alternative**.

**Étape 4 — l'algorithme.**

```

Algorithme FactureElectricite
Variables
  C, M : Reel
Debut
  Ecrire("Entrez la consommation en kWh : ")
  Lire(C)
  Si C > 110 Alors
    M ← C * 75    // tarif majoré
  Sinon
    M ← C * 50    // tarif simple
  FinSi
  Ecrire("Montant à payer : ", M, " FCFA")
Fin
  
```

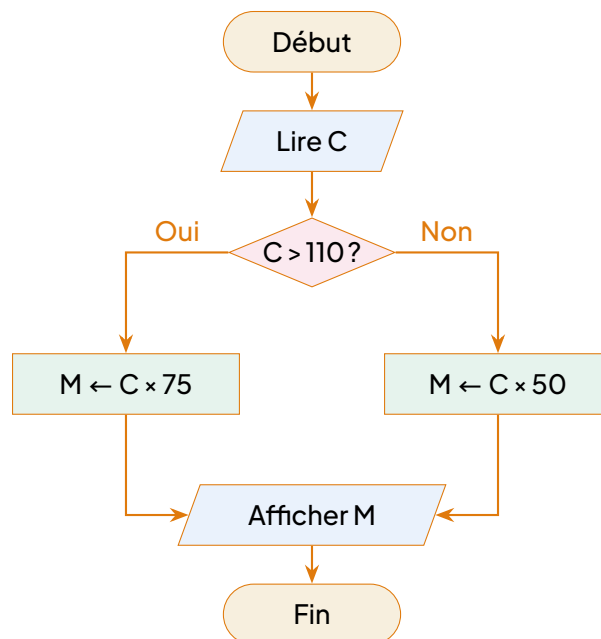
### EXEMPLE Trace complète pour une consommation de 150 kWh

Faisons « tourner » l'algorithme avec  $C = 150$  :

| Étape     | C                                | Test $C > 110$ | M                         |
|-----------|----------------------------------|----------------|---------------------------|
| Lecture   | 150                              | —              | —                         |
| Test      | 150                              | Vrai           | —                         |
| Calcul    | 150                              | —              | $150 \times 75 = 11\,250$ |
| Affichage | Montant à payer :<br>11 250 FCFA |                |                           |

Vérifions avec un second jeu de valeurs,  $C = 80$  : le test  $80 > 110$  est faux, donc  $M = 80 \times 50 = 4\,000$  FCFA. L'algorithme se comporte correctement dans les deux cas.

Voici l'algorithme correspondant, qui montre clairement la structure alternative :



Algorithme du calcul de la facture d'électricité : le losange matérialise l'alternative.

**Étape 6 — justification.** « J'ai utilisé une structure Si...Alors...Sinon car le calcul du montant dépend d'une condition (la consommation dépasse-t-elle 110 kWh?). Les deux cas sont traités de façon exclusive : le tarif majoré s'applique à toute la consommation, pas seulement au dépassement. »

### 6.3 Problème 2 : le jeu de devinette

**Situation.** L'ordinateur «pense» à un nombre secret entre 1 et 100. Le joueur propose un nombre ; tant qu'il n'a pas trouvé, on lui répond « trop grand » ou « trop petit » et on lui redemande. Décrire l'idée de l'algorithme (sans le programmer sur machine).

**Choix de la structure.** Ici, on ne sait pas à l'avance combien d'essais seront nécessaires : le joueur peut trouver du premier coup... ou au bout de dix essais. C'est donc une boucle Tant que, dont la condition d'arrêt est « le nombre proposé est égal au nombre secret ».

Algorithme Devinette

Variables

secret, proposition : Entier

Début

secret ← 42 // nombre choisi par l'ordinateur

Lire(proposition)

Tant que proposition <> secret Faire

Si proposition > secret Alors

Ecrire("Trop grand !")

Sinon

Ecrire("Trop petit !")

FinSi

Lire(proposition) // on redemande

FinTantQue

Ecrire("Bravo, trouve !")

Fin

**ATTENTION** La boucle infinie

Dans une boucle `Tant que`, il faut toujours qu'une instruction à l'intérieur de la boucle puisse rendre la condition fausse. Ici, c'est le `Lire(proposition)` placé dans la boucle qui permet de progresser. Si on l'oublie, la condition ne change jamais : la boucle ne s'arrête plus, c'est une **boucle infinie**.

**Justification type.** « J'ai choisi un `Tant que` car le nombre de répétitions est inconnu à l'avance : on répète tant que la condition "nombre non trouvé" est vraie. Un `Pour` aurait été inadapté car il exige de connaître le nombre d'itérations. »

### 6.4 Problème 3 : moyenne générale et mention

**Situation.** Un professeur veut un algorithme qui lit la moyenne générale d'un élève (sur 20) et affiche la décision : « Ajourné » si la moyenne est inférieure à 10 ; sinon « Admis », avec la mention « Passable » si la moyenne est inférieure à 12, « Bien » si elle est inférieure à 14, et « Excellent » à partir de 14.

**Choix des structures.** Il y a d'abord une grande alternative (admis ou ajourné), puis, à l'intérieur du cas « admis », d'autres tests : ce sont des **alternatives imbriquées**.

```
Algorithme Mention
Variables
    moy : Reel
Debut
    Ecrire("Entrez la moyenne : ")
    Lire(moy)
    Si moy < 10 Alors
        Ecrire("Ajourné")
    Sinon
        Ecrire("Admis")
        Si moy < 12 Alors
            Ecrire("Mention Passable")
        Sinon
            Si moy < 14 Alors
                Ecrire("Mention Bien")
            Sinon
                Ecrire("Mention Excellent")
        FinSi
    FinSi
FinSi
Fin
```

**EXERCICE RÉSOLU 6** Trace de l'algorithme « Mention »

Énoncé : tracer l'algorithme pour  $moy = 13,5$  puis pour  $moy = 8$ .

**SOLUTION**

Pour  $moy = 13,5$  : le test  $13,5 < 10$  est faux → affichage « Admis » ; le test  $13,5 < 12$  est faux ; le test  $13,5 < 14$  est vrai → affichage « Mention Bien ».

Pour  $moy = 8$  : le test  $8 < 10$  est vrai → affichage « Ajourné » ; les tests de mention ne sont jamais exécutés, car ils sont dans la branche `Sinon`.

L'algorithme est correct : chaque cas possible donne le résultat attendu. Remarque : on teste d'abord  $moy < 12$  puis  $moy < 14$  dans cet ordre, car un élève arrivé au second test a déjà une moyenne supérieure ou égale à 12.

## 6.5 Vérifier par la trace et rédiger la justification

**ATTENTION****Erreur fréquente : ne jamais tester son algorithme**

Beaucoup d'élèves écrivent un algorithme et s'arrêtent là. Or un algorithme peut sembler correct et contenir une erreur (condition inversée, mauvaise variable, cas limite oublié). Règle d'or : tester toujours sur au moins deux jeux de valeurs, dont un cas « limite » (par exemple exactement 110 kWh pour la facture, ou une moyenne de 10,00). Si la trace ne donne pas le résultat attendu, l'algorithme doit être corrigé.

La justification écrite est ce qui distingue un élève qui a compris d'un élève qui a recopié. Elle répond à trois questions : **quoi** (quelles entrées, quelles sorties), **comment** (quelles structures de contrôle), **pourquoi** (pourquoi cette structure et pas une autre). Exemple de rédaction complète :

**EXEMPLE****Modèle de justification rédigée**

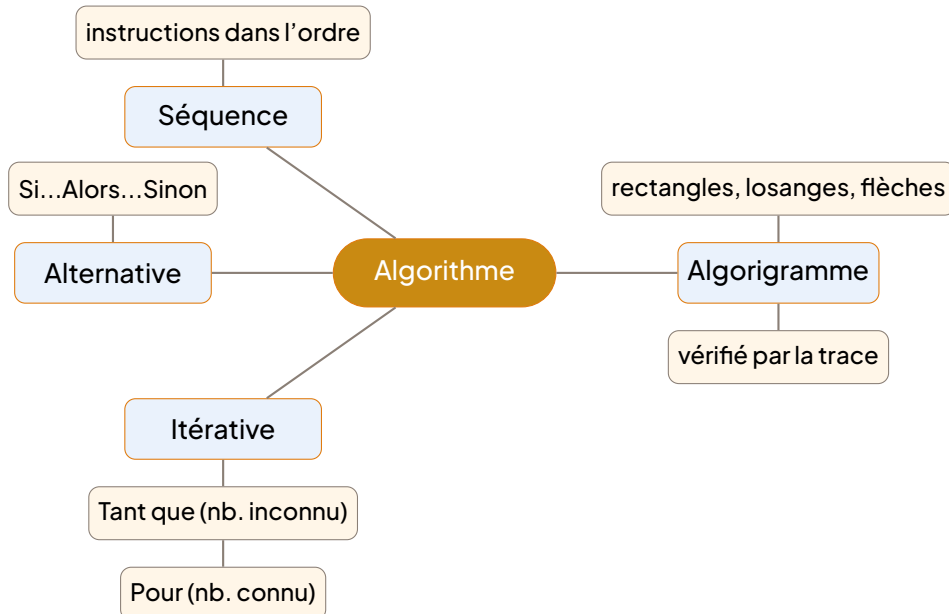
« Le problème comporte une donnée d'entrée (la consommation  $C$ ) et un résultat (le montant  $M$ ). Le calcul dépend d'une condition :  $C > 110$ . J'ai donc utilisé une structure alternative `Si...Alors...Sinon`, qui permet d'exécuter un traitement différent selon que la condition est vraie ou fausse. Aucune boucle n'est nécessaire car aucune action n'est répétée. La trace avec  $C = 150$  donne 11 250 FCFA et celle avec  $C = 80$  donne 4 000 FCFA, ce qui confirme que l'algorithme est correct. »

**LE SAVAIS-TU ?****Le conseil du correcteur**

Au BEPC, un algorithme clair, bien indenté, accompagné d'une trace d'exécution et d'une justification, vaut souvent plus de points qu'un algorithme juste mais présenté sans aucune explication. Le correcteur évalue ta démarche autant que ton résultat : montre-lui que tu sais pourquoi tu écris chaque ligne !

## 6.6 Synthèse de la leçon

Toute la leçon tient en une idée : un algorithme est une suite finie d'instructions qui combine trois « briques » — la **séquence**, l'**alternative** et l'**itérative** — et que l'on peut représenter par un algorithme.



Carte mentale de synthèse : les structures de contrôle et leur représentation.

### À RETENIR L'essentiel

- Résoudre un problème suit 6 étapes : comprendre, identifier entrées/sorties, choisir les structures, écrire, tracer, conclure en justifiant.
- Si...Alors...Sinon quand le traitement dépend d'une condition ; alternatives imbriquées pour plusieurs cas.
- Tant que quand le nombre de répétitions est inconnu ; Pour quand il est connu.
- On vérifie toujours un algorithme par une trace sur au moins deux jeux de valeurs, et on justifie ses choix par écrit.

### EXERCICE 6 À toi de jouer

Un cybercafé facture la connexion 200 FCFA l'heure ; au-delà de 3 heures, l'heure passe à 150 FCFA pour toutes les heures. Écris l'algorithme complet, trace-le pour 2 heures puis pour 5 heures, dessine son algorithme et rédige la justification de tes choix de struc-

tures.

### **CORRIGÉ** Exercice 1 : Cybercafé

#### 1. Algorithme complet

```

Algorithme FactureCyber
Variables
    duree, montant : Reel
Debut
    Ecrire("Entrez la duree de connexion (en heures) : ")
    Lire(duree)
    Si duree > 3 Alors
        montant ← duree * 150
    Sinon
        montant ← duree * 200
    FinSi
    Ecrire("Montant à payer : ", montant, " FCFA")
Fin
  
```

#### 2. Traces d'exécution

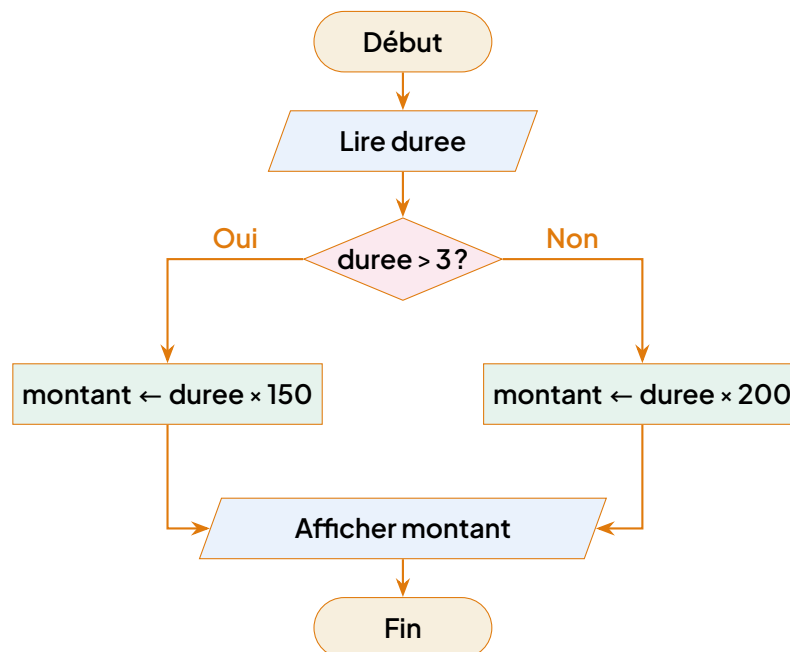
- Cas 1 :  $duree = 2$

| Étape     | duree                         | Test $duree > 3$ | montant              |
|-----------|-------------------------------|------------------|----------------------|
| Lecture   | 2                             | —                | —                    |
| Test      | 2                             | Faux             | —                    |
| Calcul    | 2                             | —                | $2 \times 200 = 400$ |
| Affichage | Montant à payer :<br>400 FCFA |                  |                      |

- Cas 2 :  $duree = 5$

| Étape     | duree                         | Test $duree > 3$ | montant              |
|-----------|-------------------------------|------------------|----------------------|
| Lecture   | 5                             | —                | —                    |
| Test      | 5                             | Vrai             | —                    |
| Calcul    | 5                             | —                | $5 \times 150 = 750$ |
| Affichage | Montant à payer : 750<br>FCFA |                  |                      |

#### 3. Algorithme



Algorithme de la facturation du cybercafé.

4. Justification « Le problème comporte une entrée (la durée de connexion) et une sortie (le montant). Le tarif change selon une condition unique : la durée dépasse-t-elle 3 heures ? J'ai donc utilisé une structure alternative *Si...Alors...Sinon*. Aucune boucle n'est nécessaire car le calcul est direct et non répétitif. Les traces pour 2 h (400 FCFA) et 5 h (750 FCFA) valident l'algorithme, y compris le cas limite de 3 h exactement (tarif normal : 600 FCFA). »

## 7 ACTIVITÉ D'INTÉGRATION

## 8 ACTIVITÉ D'INTÉGRATION : LA CAISSE DE LA COOPÉRATIVE SCOLAIRE

### 8.1 Objectifs de l'activité

À l'issue de cette activité d'intégration, tu seras capable de :

- analyser une situation réelle de gestion et identifier les **données**, les **traitements** et les **résultats** attendus ;
- choisir et **justifier** la structure de contrôle adaptée : **structure alternative** (Si ... Alors ... Sinon) pour la remise, **structure itérative** (TantQue) pour la répétition des clients ;
- écrire un algorithme complet en **pseudo-code** avec des variables correctement déclarées et initialisées ;
- représenter cet algorithme par un **algorithme** respectant les symboles conventionnels ;
- **tracer** (exécuter à la main) ton algorithme sur un jeu de données pour vérifier qu'il fonctionne, puis rédiger une conclusion argumentée.

### 8.2 Situation

**La caisse de la coopérative scolaire du Collège Bilingue de Nkongsamba.**

La coopérative scolaire du collège vend des fournitures aux élèves : cahiers, stylos, règles, tenues de classe. Chaque matin, Aminatou, élève en classe de 3<sup>ème</sup> et trésorière de la coopérative, tient la caisse. Pour fidéliser les « gros » acheteurs (souvent les parents qui achètent pour plusieurs enfants), le bureau de la coopérative a décidé la règle suivante :

Tout client dont le montant total des achats dépasse strictement 5 000 FCFA bénéficie d'une remise de 10 % sur ce montant. Les autres clients paient le montant exact.

À la fermeture de la caisse, Aminatou doit annoncer au président de la coopérative :

- la **recette totale** de la matinée (somme des montants effectivement payés, remises déduites) ;
- le **nombre de clients** servis.

Le problème, c'est qu'on ne sait **jamais à l'avance combien de clients** vont se présenter : un jour il y en a 3, un autre jour 27. Aminatou saisit donc les montants les uns après les autres, et elle signale la fin de la journée en tapant le montant 0 (un client ne peut pas avoir un montant d'achat nul : 0 est une valeur « sentinelle » qui veut dire « c'est fini »).

Ce matin-là, voici les montants des achats des clients qui se sont présentés, dans l'ordre :

| Client                    | n°1   | n°2   | n°3   | n°4    | n°5   |
|---------------------------|-------|-------|-------|--------|-------|
| Montant des achats (FCFA) | 3 500 | 8 000 | 5 000 | 12 000 | 1 200 |

Après le cinquième client, plus personne ne se présente : Aminatou saisit 0.

Fatiguée de tout calculer de tête (et de se tromper parfois!), Aminatou te demande de l'aider : **écris l'algorithme du caissier** qui, pour n'importe quelle journée, calcule et affiche la recette totale et le nombre de clients.

### 8.3 Consignes

Travail en groupes de 3 ou 4 élèves. Durée : une séance.

1. **Analyse du problème.** Identifie et liste : (a) les données que l'algorithme doit lire ; (b) les résultats qu'il doit afficher ; (c) les variables nécessaires (donne à chacune un nom et un rôle précis).
2. **Choix des structures.** Pour chacune des deux questions suivantes, choisis la structure adaptée et **justifie ton choix** en une phrase :
  - (a) Faut-il une structure alternative ? Si oui, quelle est la condition testée et que fait-on dans chaque cas ?
  - (b) Faut-il une boucle TantQue ou une boucle Pour ? Justifie précisément en t'appuyant sur le nombre de répétitions.
3. **Pseudo-code.** Rédige l'algorithme complet en pseudo-code, avec la déclaration des variables, leur initialisation, la boucle, le test, et les affichages finaux.
4. **Algorithme.** Représente ton algorithme par un algorithme en respectant les symboles : ovale (Début/Fin), parallélogramme (lecture/affichage), rectangle (traitement), losange (test).
5. **Trace d'exécution.** Exécute ton algorithme « à la main » sur le jeu des 5 montants du tableau ci-dessus. Présente un tableau donnant, à chaque étape : le montant lu, la remise éventuelle, le montant payé, la recette cumulée et le nombre de clients. Vérifie que les résultats finaux sont cohérents.
6. **Justification orale.** Prépare une réponse argumentée (3 ou 4 phrases) à la question du professeur : « Pourquoi avez-vous choisi la boucle TantQue plutôt que la boucle Pour ? »
7. **Pour aller plus loin (bonus).** Modifie ton algorithme pour qu'il affiche aussi, à la fin, le **montant moyen** payé par client. Attention au cas où aucun client ne s'est présenté !

### 8.4 Ressources à mobiliser

#### À RETENIR

#### Ce dont tu as besoin pour réussir

- **Structure alternative** : Si condition Alors traitement1 Sinon traitement2 FinSi.  
La condition ici est une comparaison : `montant > 5000`.

- Calcul d'une remise de 10 % :  $\text{montant payé} = \text{montant} - \text{montant} \times \frac{10}{100}$ , c'est-à-dire  $\text{montant payé} = \text{montant} \times 0,9$ .
- Boucle TantQue : `TantQue condition Faire ... FinTantQue`. On l'utilise quand on ne connaît pas à l'avance le nombre de répétitions ; la condition est testée avant chaque tour.
- Boucle Pour : réservée au cas où le nombre de tours est connu à l'avance.
- Accumulateur : pour cumuler une recette, on initialise une variable à 0 puis on ajoute :  $\text{recette} \leftarrow \text{recette} + \text{montantPayé}$ .
- Compteur : pour compter les clients :  $\text{nbClients} \leftarrow \text{nbClients} + 1$ .
- Lecture avant la boucle : avec une valeur sentinelle, il faut lire une première fois avant le TantQue, puis relire à la fin du corps de la boucle.
- Symboles de l'algorithme : ovale = début/fin ; parallélogramme = entrée/sortie ; rectangle = traitement ; losange = test (deux flèches sortantes : Oui/Non).

## 8.5 Démarche et corrigé commenté

### EXERCICE RÉSOLU 7 La caisse de la coopérative scolaire

Énoncé : écrire en pseudo-code puis en algorithme l'algorithme du caissier de la coopérative (remise de 10 % au-delà de 5 000 FCFA, arrêt sur la saisie de 0), le tracer sur les montants 3 500 ; 8 000 ; 5 000 ; 12 000 ; 1 200, et justifier le choix de la boucle.

#### SOLUTION

Étape 1 : analyse du problème.

- Données lues : les montants des achats, un par un, jusqu'à la saisie de 0.
- Résultats affichés : la recette totale et le nombre de clients.
- Variables :
  - `montant` : montant des achats du client en cours (réel) ;
  - `montantPayé` : montant effectivement payé après remise éventuelle (réel) ;
  - `recette` : accumulateur de la recette totale, initialisé à 0 (réel) ;
  - `nbClients` : compteur de clients, initialisé à 0 (entier).

Étape 2 : choix et justification des structures.

1. Structure alternative : oui, il en faut une, car le traitement dépend du montant. La condition est `montant > 5000` : si elle est vraie, on applique la remise de 10 % ; sinon,

le client paie le montant exact. Attention : « dépasse 5 000 » signifie strictement supérieur ; un montant de exactement 5 000 FCFA ne donne pas droit à la remise.

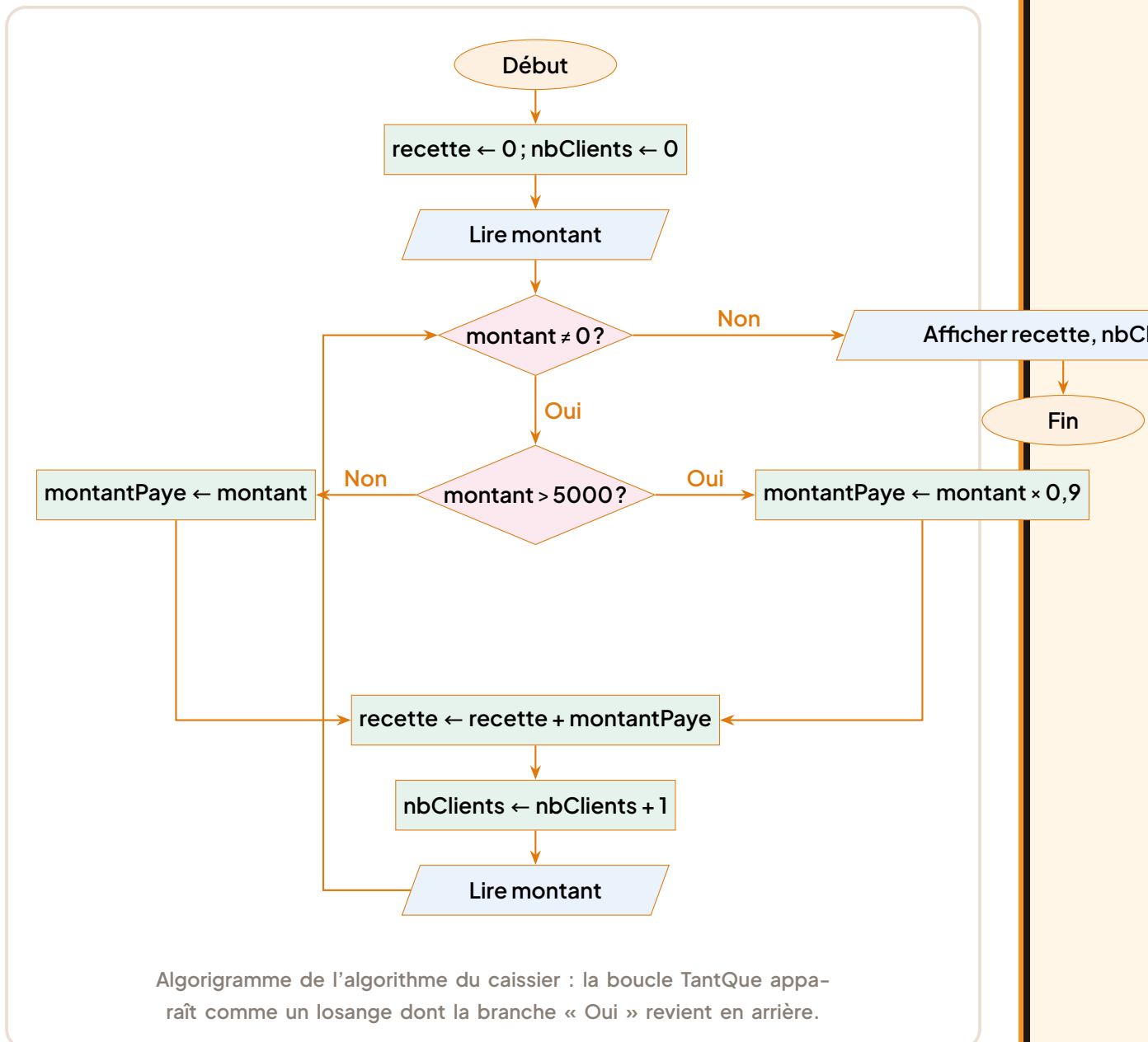
2. Boucle : on choisit TantQue, car le nombre de clients n'est pas connu à l'avance : la répétition dépend d'une condition (`montant <> 0`) et non d'un compteur de tours fixé au départ. La boucle Pour est impossible ici, puisqu'elle exige de connaître le nombre de répétitions avant de commencer.

Étape 3 : le pseudo-code.

```
Algorithme CaisseCooperative
Variables
    montant, montantPaye, recette : Reel
    nbClients : Entier
Debut
    recette <- 0
    nbClients <- 0
    Ecrire("Entrez le montant des achats (0 pour terminer) : ")
    Lire(montant)
    TantQue montant <> 0 Faire
        Si montant > 5000 Alors
            montantPaye <- montant * 0.9 // remise de 10 %
        Sinon
            montantPaye <- montant // pas de remise
        FinSi
        recette <- recette + montantPaye
        nbClients <- nbClients + 1
        Ecrire("Entrez le montant des achats (0 pour terminer) : ")
        Lire(montant)
    FinTantQue
    Ecrire("Recette totale : ", recette, " FCFA")
    Ecrire("Nombre de clients : ", nbClients)
Fin
```

Deux points délicats à remarquer : la première lecture a lieu avant la boucle (sinon la condition du TantQue porterait sur une variable vide), et la lecture suivante est placée à la fin du corps de la boucle (sinon on bouclerait indéfiniment sur le même montant).

Étape 4 : l'algorithme.



Étape 5 : trace d'exécution sur le jeu de montants.

| Étape          | Montant lu | Test<br>> 5 000?                           | Montant<br>payé | Recette cu-<br>mulée | nbClients |
|----------------|------------|--------------------------------------------|-----------------|----------------------|-----------|
| Initialisation | —          | —                                          | —               | 0                    | 0         |
| Client 1       | 3 500      | Non                                        | 3 500           | 3 500                | 1         |
| Client 2       | 8 000      | Oui                                        | 7 200           | 10 700               | 2         |
| Client 3       | 5 000      | Non                                        | 5 000           | 15 700               | 3         |
| Client 4       | 12 000     | Oui                                        | 10 800          | 26 500               | 4         |
| Client 5       | 1 200      | Non                                        | 1 200           | 27 700               | 5         |
| Fin            | 0          | Sortie de boucle : affichage des résultats |                 |                      |           |

Détail des remises :  $8\,000 \times 0,9 = 7\,200$  FCFA et  $12\,000 \times 0,9 = 10\,800$  FCFA. Le client 3, avec exactement 5 000 FCFA, ne dépasse pas 5 000 : il paie donc 5 000 FCFA, sans remise. L'algorithme affiche finalement :

Recette totale : 27 700 FCFA — Nombre de clients : 5

Étape 6 : justification orale du choix de la boucle.

« Nous avons choisi la boucle TantQue parce que le nombre de clients n'est pas connu à l'avance : il varie chaque jour. La boucle Pour exige de fixer le nombre de tours avant de commencer, ce qui est impossible ici. Avec TantQue, la répétition est contrôlée par la condition *montant* <> 0 : dès qu'Aminatou saisit la valeur sentinelle 0, la condition devient fausse et la boucle s'arrête. C'est donc la structure adaptée à une répétition dont le nombre de tours dépend d'un événement. »

Étape 7 (bonus) : montant moyen par client.

Il suffit d'ajouter, après la boucle, un test pour éviter la division par zéro :

```
Si nbClients > 0 Alors
    Ecrire("Montant moyen : ", recette / nbClients, " FCFA")
Sinon
    Ecrire("Aucun client ce jour.")
FinSi
```

Sur notre exemple :  $27\,700 \div 5 = 5\,540$  FCFA en moyenne par client.

#### ATTENTION Erreurs fréquentes à éviter

- Oublier la lecture avant la boucle : la condition du TantQue porterait sur une variable non initialisée.
- Oublier la lecture à la fin du corps de boucle : le programme traite le même montant indéfiniment (boucle infinie).
- Écrire *montant* >= 5000 au lieu de *montant* > 5000 : le client à exactement 5 000 FCFA

recevrait la remise à tort. Lis bien l'énoncé : « dépasse » signifie strictement supérieur.

- Cumuler `montant` au lieu de `montantPaye` : la recette serait fausse dès qu'une remise s'applique.
- Compter la sentinelle : le 0 final ne doit ni être ajouté à la recette ni être compté comme un client ; c'est bien le test du `TantQue` qui l'écarte.

## 8.6 Bilan de l'activité

Cette activité t'a permis de mettre en évidence plusieurs idées essentielles de la leçon :

- **Le choix d'une structure de contrôle se justifie** : on ne choisit pas une boucle « au hasard ». La boucle **TantQue** s'impose quand le nombre de répétitions est inconnu et dépend d'une condition (ici, la saisie de la valeur sentinelle 0) ; la boucle **Pour** est réservée aux répétitions dont le nombre de tours est connu à l'avance ; la structure **alternative** traduit une décision (accorder ou non la remise).
- **Un algorithme de gestion réel** repose souvent sur le trio lecture – test – cumul : lire une donnée, décider du traitement, accumuler dans un **accumulateur** et compter avec un **compteur**.
- **L'algorithme** est un outil de communication : il rend la logique de l'algorithme visible (la boucle apparaît comme une flèche qui « remonte ») et permet de vérifier la démarche avant de la rédiger en pseudo-code.
- **La trace d'exécution** est une méthode de vérification rigoureuse : en exécutant l'algorithme à la main sur un jeu de données connu, on détecte les erreurs (remise mal calculée, sentinelle comptée, boucle infinie) avant toute utilisation réelle.
- Enfin, tu as rédigé et justifié une démarche complète, de l'analyse du problème jusqu'à la conclusion argumentée : c'est exactement la compétence attendue au BEPC.

## 9 MÉTHODES ET EXERCICES RÉSOLUS

## 10 UTILISATION DES STRUCTURES DE CONTRÔLE

### 10.1 Méthode 1 : Écrire un algorithme simple en pseudo-code

#### MÉTHODE Rédiger un algorithme proprement

Pour écrire un algorithme simple qui résout un problème de la vie courante :

1. Lire l'énoncé et identifier ce que l'on connaît (les **données**) et ce que l'on cherche (le **résultat**).
2. Déclarer les variables : donner un nom explicite et un **type** (Entier, Reel, **Chaîne de caractères**) à chaque donnée et à chaque résultat.
3. Écrire l'en-tête : le mot `Algorithme` suivi d'un nom significatif, puis la liste des variables.
4. Saisir les données avec l'instruction `Lire` (ou `Saisir`).
5. Décrire les traitements dans l'ordre logique, avec des **affectations** du type `variable ← expression`.
6. Afficher le résultat avec l'instruction `Écrire` (ou `Afficher`).
7. Terminer par le mot `Fin` et vérifier l'algorithme en le testant « à la main » avec des valeurs simples (trace d'exécution).

Réflexes à retenir : une variable doit toujours être déclarée avant d'être utilisée ; l'affectation se lit de droite à gauche (`a ← 5` signifie « a reçoit la valeur 5 ») ; chaque instruction occupe une ligne ; on écrit les mots-clés en français (`Debut`, `Fin`, `Si`, `Alors`, `TantQue`, `Pour`).

#### EXERCICE RÉSOLU 8 Le taxi de Ngoa-Ekellé

Énoncé. Un taximan de Yaoundé facture une course de la façon suivante : une prise en charge fixe de 200 FCFA, plus 150 FCFA par kilomètre parcouru. Écrire un algorithme qui demande le nombre de kilomètres parcourus et affiche le prix total de la course.

#### SOLUTION

Solution détaillée.

Étape 1 — Analyse. Donnée connue : le nombre de kilomètres, noté `km` (réel). Résultat

cherché : le prix total, noté `prix` (réel). La formule est :

$$\text{prix} = 200 + 150 \times \text{km}$$

Étape 2 — Rédaction de l'algorithme.

```
Algorithme PrixCourse
Variables
  km, prix : Reel
Debut
  Ecrire("Entrez le nombre de kilometres : ")
  Lire(km)
  prix ← 200 + 150 * km
  Ecrire("Le prix de la course est : ", prix, " FCFA")
Fin
```

Étape 3 — Vérification. Testons avec `km = 4` : `prix = 200 + 150 × 4 = 200 + 600 = 800 FCFA`. L'algorithme affiche bien 800 FCFA, ce qui correspond au calcul manuel.

Conclusion. L'algorithme `PrixCourse` saisit la distance, applique la formule `prix = 200 + 150 km` et affiche le montant à payer : il résout correctement le problème posé.

## 10.2 Méthode 2 : Utiliser une structure alternative (Si ... Alors ... Sinon)

### MÉTHODE Choisir et rédiger un test

Quand le traitement dépend d'une **condition** :

1. Identifier la condition : c'est une question dont la réponse est VRAI ou FAUX (comparaison avec `=`, `<`, `>`, `≤`, `≥`, `≠`). On peut combiner des conditions avec `Et`, `Ou`, `Non`.
2. Choisir la forme adaptée :
  - Alternative simple (un seul cas à traiter) : `Si condition Alors ... FinSi`
  - Alternative complète (deux cas) : `Si condition Alors ... Sinon ... FinSi`
  - Cas multiples : enchaîner avec `SinonSi`.
3. Écrire les instructions de chaque branche, en les décalant (indentation) pour la lisibilité.
4. Fermer le test avec `FinSi` : oublier ce mot est une erreur classique.
5. Tester les deux branches avec des valeurs qui rendent la condition vraie, puis fausse (dont les valeurs limites).

Attention aux bornes : « au moins 10 » se traduit par `note >= 10` et non `note > 10` ; « au plus 10 » par `note <= 10`.

**EXERCICE RÉSOLU 9** Décision du conseil de classe

Énoncé. Dans un collège de Douala, un élève de 3ème est déclaré admis en classe supérieure si sa moyenne annuelle est supérieure ou égale à 10/20. Écrire un algorithme qui demande la moyenne d'un élève et affiche « Admis » ou « Ajourné ».

**SOLUTION**

Solution détaillée.

Étape 1 — Analyse. Donnée : la moyenne  $m$  (réel, comprise entre 0 et 20). Condition :  $m \geq 10$ . Deux cas possibles, donc alternative complète.

Étape 2 — Rédaction.

```
Algorithme DecisionConseil
Variables
  m : Reel
Debut
  Ecrire("Entrez la moyenne annuelle : ")
  Lire(m)
  Si m >= 10 Alors
    Ecrire("Admis en classe superieure")
  Sinon
    Ecrire("Ajourne")
  FinSi
Fin
```

Étape 3 — Vérification des deux branches.

- Avec  $m = 12,5$  : la condition  $12,5 \geq 10$  est VRAIE, l'algorithme affiche « Admis ». Correct.
- Avec  $m = 10$  : la condition  $10 \geq 10$  est VRAIE, l'algorithme affiche « Admis ». C'est bien le cas limite demandé (« supérieure ou égale à 10 »).
- Avec  $m = 7$  : la condition est FAUSSE, l'algorithme affiche « Ajourné ». Correct.

Conclusion. L'alternative complète `Si ... Alors ... Sinon ... FinSi` permet de traiter les deux décisions possibles ; le test sur la valeur limite 10 confirme que l'opérateur  $\geq$  était le bon choix.

**10.3 Méthode 3 : Utiliser la boucle « Tant que »****MÉTHODE** Construire une boucle Tant que

On utilise `TantQue` quand on ne connaît pas à l'avance le nombre de répétitions :

1. Identifier la condition de continuation : on répète tant que la condition est vraie ; on s'arrête dès qu'elle devient fausse.
2. Initialiser avant la boucle les variables utilisées dans la condition (sinon le premier test est impossible).

3. Écrire le corps de la boucle : les instructions à répéter.
4. Prévoir la modification : dans le corps, au moins une instruction doit faire évoluer la condition (par exemple `Lire` ou une incrémentation), sinon la boucle est infinie.
5. Fermer avec `FinTantQue` et tracer l'exécution dans un tableau (une colonne par variable, une ligne par tour de boucle).

Point de vigilance : si la condition est fausse dès le départ, le corps de la boucle n'est jamais exécuté : c'est normal et parfois souhaité.

#### EXERCICE RÉSOLU 10 L'épargne de la tontine

Énoncé. Amina dépose chaque mois 5 000 FCFA sur son compte épargne, qui contient initialement 12 000 FCFA. Elle veut savoir au bout de combien de mois son épargne atteindra ou dépassera 50 000 FCFA. Écrire un algorithme qui calcule et affiche ce nombre de mois.

#### SOLUTION

Solution détaillée.

Étape 1 — Analyse. On ne connaît pas à l'avance le nombre de dépôts : c'est une boucle `TantQue`. Variables : `solde` (réel) initialisé à 12 000, `mois` (entier) initialisé à 0. Condition de continuation : `solde < 50000` (on continue tant que l'objectif n'est pas atteint).

Étape 2 — Rédaction.

Algorithme EpargneTontine

Variables

solde : Reel

mois : Entier

Debut

solde ← 12000

mois ← 0

TantQue solde < 50000 Faire

solde ← solde + 5000

mois ← mois + 1

FinTantQue

Ecrire("Objectif atteint au bout de ", mois, " mois")

Fin

Étape 3 — Trace d'exécution.

| Tour | solde (avant test) | mois | Condition $\text{solde} < 50000$ |
|------|--------------------|------|----------------------------------|
| 0    | 12 000             | 0    | VRAI                             |
| 1    | 17 000             | 1    | VRAI                             |
| 2    | 22 000             | 2    | VRAI                             |
| ...  | ...                | ...  | VRAI                             |
| 7    | 47 000             | 7    | VRAI                             |
| 8    | 52 000             | 8    | FAUX : on sort                   |

Le solde augmente de 5 000 par tour ; après 8 tours,  $\text{solde} = 12\,000 + 8 \times 5\,000 = 52\,000 \geq 50\,000$  : la boucle s'arrête.

Conclusion. L'algorithme affiche « Objectif atteint au bout de 8 mois ». La boucle `TantQue` était bien adaptée car le nombre de répétitions était inconnu au départ.

#### 10.4 Méthode 4 : Utiliser la boucle « Pour »

##### MÉTHODE Construire une boucle Pour

On utilise `Pour` quand le nombre de répétitions est connu à l'avance :

1. Déterminer le compteur : une variable entière (souvent `i`) qui va parcourir les valeurs.
2. Déterminer les bornes : valeur de début et valeur de fin (la borne de fin est incluse).
3. Écrire la syntaxe : `Pour i de 1 à N Faire ... FinPour`. Le compteur augmente automatiquement de 1 à chaque tour : il ne faut pas l'incrémenter soi-même.
4. Initialiser avant la boucle les variables d'accumulation ( $\text{somme} \leftarrow 0$ ,  $\text{produit} \leftarrow 1$ ).
5. Vérifier le nombre de tours : de  $a$  à  $b$ , il y a exactement  $b - a + 1$  répétitions.

Réflexe : pour calculer une somme  $S = 1 + 2 + \dots + N$ , on écrit dans la boucle  $S \leftarrow S + i$ .

##### EXERCICE RÉSOLU 11 Total des ventes de la semaine

Énoncé. La boutique de la maman de Junior est ouverte 6 jours par semaine. Chaque jour, on note la recette du jour. Écrire un algorithme qui demande les 6 recettes, puis calcule et affiche la recette totale de la semaine.

##### SOLUTION

Solution détaillée.

Étape 1 — Analyse. Le nombre de saisies est connu : 6 jours, donc 6 répétitions  $\Rightarrow$  boucle `Pour`. Variables : `recette` (réel) pour chaque saisie, `total` (réel) accumulateur initialisé à 0,

`i` (entier) compteur.

Étape 2 — Rédaction.

```
Algorithme RecetteSemaine
Variables
  recette, total : Reel
  i : Entier
Debut
  total <- 0
  Pour i de 1 a 6 Faire
    Ecrire("Recette du jour", i, " : ")
    Lire(recette)
    total <- total + recette
  FinPour
  Ecrire("Recette totale de la semaine : ", total, " FCFA")
Fin
```

Étape 3 — Vérification. Nombre de tours :  $6-1+1 = 6$ , donc 6 saisies, comme prévu. Testons avec six recettes de 10 000 FCFA chacune : après chaque tour, `total` vaut successivement 10 000 ; 20 000 ; 30 000 ; 40 000 ; 50 000 ; 60 000. L'algorithme affiche 60 000 FCFA, ce qui est exact :  $6 \times 10\,000 = 60\,000$ .

Conclusion. La boucle `Pour` convient parfaitement car le nombre de jours (6) est fixe ; l'accumulateur `total`, initialisé à 0 avant la boucle, contient bien la somme des 6 recettes à la sortie.

## 10.5 Méthode 5 : Représenter un algorithme par un algorithme

### MÉTHODE Dessiner un algorithme correct

Pour traduire un algorithme en schéma graphique normé :

1. Respecter les symboles normalisés :
  - Ovale : Début / Fin ;
  - Parallélogramme : entrée (Lire) ou sortie (Écrire) ;
  - Rectangle : traitement (affectation, calcul) ;
  - Losange : test (condition), avec deux flèches sortantes étiquetées Oui / Non.
2. Commencer par un ovale « Début » et terminer par un ovale « Fin ».
3. Relier les symboles par des flèches orientées de haut en bas, dans l'ordre d'exécution.
4. Pour une alternative : le losange a deux sorties ; les deux branches se rejoignent (ou vont vers Fin) après leurs instructions.
5. Pour une boucle : la flèche « Oui » du losange descend vers le corps de la boucle, puis une flèche remonte vers le losange ; la flèche « Non » continue vers la suite.

6. Vérifier que chaque losange a exactement deux sorties et que tout chemin mène à « Fin ».

**EXERCICE RÉSOLU 12** Algorithme du plus grand de deux nombres

Énoncé. Écrire l'algorithme qui lit deux nombres et affiche le plus grand, puis le représenter par un algorithme.

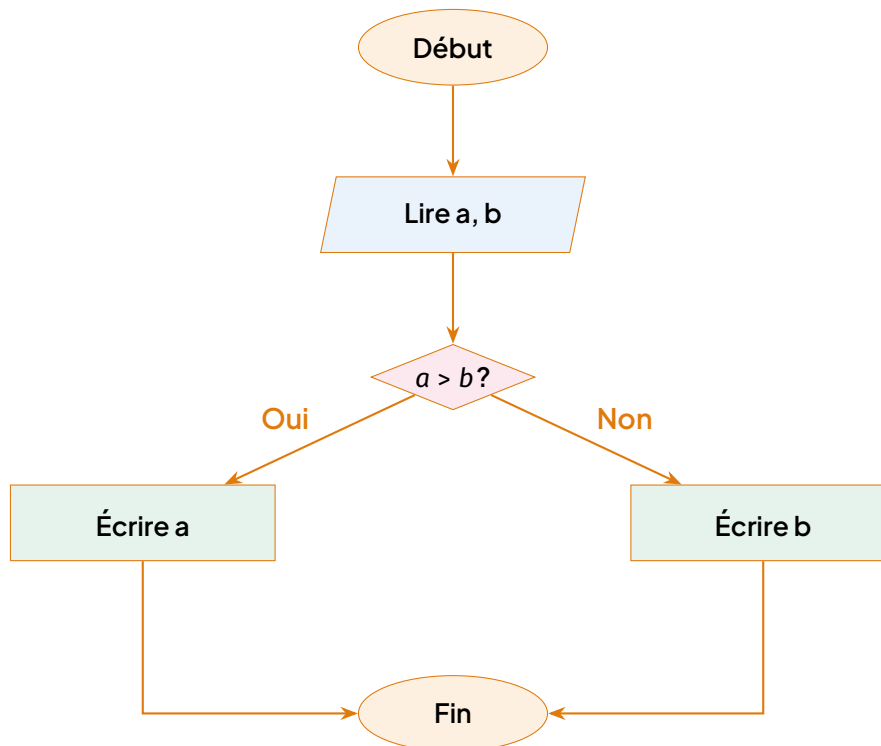
**SOLUTION**

Solution détaillée.

Étape 1 — Algorithme.

```
Algorithme PlusGrand
Variables
  a, b : Reel
Debut
  Lire(a)
  Lire(b)
  Si a > b Alors
    Ecrire("Le plus grand est :", a)
  Sinon
    Ecrire("Le plus grand est :", b)
  FinSi
Fin
```

Étape 2 — Algorithme.



Algorigramme de l'algorithme `PlusGrand` : ovale (début/fin), parallélogramme (lecture), losange (test à deux sorties), rectangles (affichages).

**Étape 3 — Vérification.** Le losange possède bien deux sorties étiquetées « Oui » et « Non » ; les deux chemins se rejoignent avant « Fin ». Test : avec  $a = 7$  et  $b = 3$ , la condition  $7 > 3$  est vraie, on affiche 7 ; avec  $a = 2$  et  $b = 9$ , elle est fausse, on affiche 9. Les deux branches fonctionnent.

**Conclusion.** L'algorigramme traduit fidèlement l'algorithme : chaque symbole normalisé correspond à une instruction, et les flèches décrivent l'ordre d'exécution.

## 10.6 Méthode 6 : Résoudre un problème concret et justifier sa démarche

### MÉTHODE Rédiger et justifier une solution complète

Face à un problème de la vie courante à résoudre par un algorithme (situation type BEPC) :

1. Dégager les données et les résultats dans une petite phrase d'analyse.
2. Choisir les structures et le justifier : « j'utilise un `Si` car il y a un choix », « j'utilise un `Pour` car le nombre de répétitions est connu », « j'utilise un `TantQue` car on répète jusqu'à

atteindre un objectif ».

3. Rédiger l'algorithme complet : en-tête, déclarations, Debut ... Fin.
4. Valider par une trace d'exécution : tableau donnant la valeur de chaque variable à chaque étape, avec des valeurs numériques de l'énoncé.
5. Conclure par une phrase qui répond exactement à la question posée, avec l'unité (FCFA, mois, élèves...).

Barème implicite : la justification du choix des structures et la trace d'exécution rapportent souvent autant de points que l'algorithme lui-même.

### EXERCICE RÉSOLU 13 La cantine du collège

**Énoncé.** Le cantinier d'un collège de Bafoussam vend le ticket de repas à 500 FCFA. Un élève ne peut acheter un ticket que s'il présente sa carte et si le stock n'est pas épuisé. Le stock initial est de 100 tickets. Écrire un algorithme qui, pour chacun des élèves d'une file de  $N$  élèves, demande s'il a sa carte (1 = oui, 0 = non), vend le ticket si possible, et affiche à la fin la recette totale et le nombre de tickets restants.

#### SOLUTION

Solution détaillée.

Étape 1 — Analyse. Données :  $N$  (nombre d'élèves, entier), la réponse `carte` (0 ou 1) pour chaque élève. Résultats : `recette` et `stock`.

Étape 2 — Choix justifié des structures. Le nombre d'élèves  $N$  est connu à l'avance : boucle `Pour`. À chaque élève, il y a une décision (vendre ou non) : alternative `Si`. La double condition « carte présentée et stock disponible » se traduit par `carte = 1 Et stock > 0`.

Étape 3 — Algorithme.

```

Algorithme Cantine
Variables
    N, i, carte, stock : Entier
    recette : Reel
Debut
    stock <- 100
    recette <- 0
    Ecrire("Nombre d'eleves dans la file : ")
    Lire(N)
    Pour i de 1 a N Faire
        Ecrire("L'eleve ", i, " a-t-il sa carte ? (1=oui, 0=non) ")
        Lire(carte)
        Si carte = 1 Et stock > 0 Alors
            stock <- stock - 1
            recette <- recette + 500
            Ecrire("Ticket vendu")
        Sinon
            Ecrire("Vente refusee")
        FinSi
    FinPour
Fin
  
```

```

FinPour
Ecrire("Recette totale : ", recette, " FCFA")
Ecrire("Tickets restants : ", stock)
Fin

```

Étape 4 — Trace d'exécution avec  $N = 3$  et les réponses 1, 0, 1 (stock initial 100) :

| Tour $i$ | carte | stock | recette | Action                                  |
|----------|-------|-------|---------|-----------------------------------------|
| 1        | 1     | 99    | 500     | Vente acceptée ( $1 = 1$ et $100 > 0$ ) |
| 2        | 0     | 99    | 500     | Vente refusée (pas de carte)            |
| 3        | 1     | 98    | 1 000   | Vente acceptée                          |

Étape 5 — Conclusion. Pour cette file de 3 élèves, l'algorithme affiche une recette de 1 000 FCFA et 98 tickets restants, ce qui correspond exactement aux deux ventes réalisées. La démarche est justifiée : boucle `Pour` car  $N$  est connu, alternative avec condition double car la vente exige à la fois la carte et un stock non nul.

#### À RETENIR Synthèse : Les trois structures de contrôle

- Structure séquentielle : instructions exécutées les unes après les autres (par défaut).
- Structure alternative (Choix) : `Si condition Alors ... Sinon ... FinSi`. Exécute un bloc ou l'autre selon une condition.
- Structure itérative (Boucle) :
  - Pour  $i$  de  $a$  à  $b$  Faire ... FinPour : nombre de tours connu ( $b - a + 1$  tours). Compteur auto-incrémenté.
  - TantQue condition Faire ... FinTantQue : nombre de tours inconnu. Condition testée avant chaque tour. Initialisation obligatoire avant la boucle.

Règle d'or : Toute variable utilisée (compteur, accumulateur, condition) doit être initialisée avant d'entrer dans la structure.

#### ATTENTION Les 5 erreurs qui coûtent des points à l'examen

1. Confondre «  $>$  » et «  $\geq$  ». « Au moins 10/20 » se traduit par  $m \geq 10$ . Avec  $m > 10$ , un élève ayant exactement 10 est rejeté : la condition d'application est fausse et la réponse au cas limite est incorrecte.
2. Oublier d'initialiser les variables avant la boucle. Un accumulateur non initialisé (total ou solde) a une valeur inconnue : tous les résultats suivants sont faux. Toujours écrire `total <- 0` (ou la valeur initiale de l'énoncé) avant la boucle.

3. Créer une boucle infinie avec « TantQue ». Si aucune instruction du corps ne fait évoluer la condition (oubli de `mois <- mois + 1`), la boucle ne s'arrête jamais. Vérifier toujours que la condition finira par devenir fausse.
4. Incrémenter soi-même le compteur d'une boucle « Pour ». Dans `Pour i de 1 à N`, le compteur augmente automatiquement ; ajouter `i <- i + 1` dans le corps fausse le nombre de tours (il y a  $b - a + 1$  tours, pas plus).
5. Mal dessiner l'algorithme. Erreurs fréquentes : utiliser un rectangle pour un test (il faut un losange), oublier les étiquettes « Oui/Non » sur les deux flèches sortantes, ou faire remonter la flèche d'une boucle au mauvais endroit. Chaque losange doit avoir exactement deux sorties et tout chemin doit aboutir à « Fin ».

**EXERCICE 7** Entraînement 1 : Pair ou Impair

Écrire un algorithme qui demande un entier positif à l'utilisateur et affiche « Pair » s'il est divisible par 2, « Impair » sinon. Représenter l'algorithme par un algorithme.

**EXERCICE 8** Entraînement 2 : Table de multiplication

Écrire un algorithme qui demande un entier  $n$  et affiche sa table de multiplication de 1 à 10 (ex : «  $1 \times 7 = 7$  », «  $2 \times 7 = 14$  », ...). Justifier le choix de la boucle.

**EXERCICE 9** Entraînement 3 : Saisie sécurisée

Écrire un algorithme qui demande à l'utilisateur de saisir une note comprise entre 0 et 20. Tant que la note n'est pas dans cet intervalle, il redemande la saisie. Une fois la note valide, il l'affiche. Justifier le choix de la boucle.

## 11 EXERCICES

### 11.1 Je vérifie mes connaissances

#### EXERCICE 10 QCM : structures de contrôle

Pour chaque question, entoure la (ou les) bonne(s) réponse(s).

1. Un algorithme est :
  - (a) un langage de programmation ;
  - (b) une suite finie et ordonnée d'instructions permettant de résoudre un problème ;
  - (c) un composant de l'unité centrale.
2. La structure alternative simple s'écrit en pseudo-code :
  - (a) Si ... Alors ... Sinon ... FinSi ;
  - (b) TantQue ... Faire ... FinTantQue ;
  - (c) Lire ... Ecrire.
3. La boucle `Pour` est utilisée lorsque :
  - (a) le nombre de répétitions est connu à l'avance ;
  - (b) la condition d'arrêt dépend d'un calcul imprévisible ;
  - (c) on veut lire une donnée au clavier.
4. Dans un algorithme, un test (une décision) se représente par :
  - (a) un rectangle ;
  - (b) un losange ;
  - (c) un ovale.

#### EXERCICE 11 Vrai ou faux ? Justifie

Réponds par Vrai ou Faux, puis justifie ta réponse en une phrase.

1. Une boucle `TantQue` peut ne jamais s'exécuter si sa condition est fausse dès le départ.
2. Dans une boucle `TantQue`, il n'est pas nécessaire de modifier la variable de la condition à l'intérieur de la boucle.
3. Un algorithme commence et se termine toujours par un ovale.
4. L'instruction `Sinon` est obligatoire dans toute structure alternative.

**EXERCICE 12 Définitions**

Définis brièvement les termes suivants et donne un exemple pour chacun :

1. algorithme ;
2. structure alternative ;
3. structure itérative (boucle) ;
4. algorithme.

**EXERCICE 13 Trace d'exécution**

On donne l'algorithme suivant :

```
Algorithme Mystere
Variable n, i, s : Entier
Debut
  Lire(n)
  s <- 0
  Pour i allant de 1 a n Faire
    s <- s + i
  FinPour
  Ecrire(s)
Fin
```

1. Recopie et complète la table d'exécution ci-dessous pour  $n = 4$  :

| Étape                | Valeur de i | Valeur de s |
|----------------------|-------------|-------------|
| Avant la boucle      | -           |             |
| 1 <sup>er</sup> tour | 1           |             |
| 2 <sup>e</sup> tour  | 2           |             |
| 3 <sup>e</sup> tour  | 3           |             |
| 4 <sup>e</sup> tour  | 4           |             |

2. Quelle valeur l'algorithme affiche-t-il ? Que calcule cet algorithme en général ?

**11.2 Je m'entraîne****EXERCICE 14 Mineur ou majeur**

1. Écris en pseudo-code un algorithme qui lit l'âge d'un élève et affiche « Mineur » si l'âge est strictement inférieur à 18 ans, et « Majeur » sinon.

- Trace l'exécution de ton algorithme pour un élève de 15 ans, puis pour un élève de 19 ans (indique la valeur du test et le message affiché dans chaque cas).

**EXERCICE 15** Somme des nombres pairs

- Écris en pseudo-code un algorithme qui calcule et affiche la somme des nombres pairs de 2 à 50, en utilisant une boucle `Pour`.
- Donne la trace partielle de l'exécution : valeurs de la variable de boucle et de la somme pour les trois premiers tours et pour le dernier tour.
- Quelle valeur finale est affichée ?

**EXERCICE 16** Le vendeur de crédit téléphonique

Un vendeur de crédit téléphonique de quartier part de 0 FCFA d'économies et met de côté 2 500 FCFA chaque jour. Il veut savoir au bout de combien de jours il disposera d'au moins 30 000 FCFA pour acheter une mallette de recharge.

- Écris en pseudo-code un algorithme utilisant une boucle `TantQue` qui calcule et affiche le nombre de jours nécessaires.
- Vérifie par le calcul le résultat affiché par ton algorithme.
- Que se passerait-il si l'on oubliait l'instruction qui augmente le nombre de jours dans la boucle ?

**EXERCICE 17** Le plus grand de deux nombres

- Écris en pseudo-code un algorithme qui lit deux nombres réels  $a$  et  $b$  et affiche le plus grand des deux.
- Représente cet algorithme par un algorithme (ovales, rectangles, losange et flèches).
- Que faudrait-il ajouter pour traiter correctement le cas où les deux nombres sont égaux ?

### 11.3 Je me prépare au Probatoire

**EXERCICE 18** Chasse aux erreurs

Le directeur d'une école demande à un élève de 3<sup>e</sup> un algorithme qui lit un nombre entier  $n$  et affiche la somme  $1+2+\dots+n$ . L'élève propose le pseudo-code et l'algorithme (décrit ci-dessous, qui contiennent trois erreurs :

```
Algorithme Somme
Variable n, i, s : Entier
Debut
  Lire(n)
  s <- 0
  i <- 1
  TantQue i <= n Faire
    s <- s + i
  FinTantQue
  Si s > 100 Alors
    Ecrire("Grande somme")
  Ecrire(s)
Fin
```

L'algorithme tracé par l'élève représente le test  $i \leq n$  par un rectangle au lieu de la forme correcte.

1. Releve les trois erreurs commises (une boucle infinie, un `FinSi` manquant, un mauvais symbole dans l'algorithme) en expliquant pour chacune pourquoi c'est une erreur.
2. Écris le pseudo-code corrigé.
3. Indique le symbole correct pour représenter le test dans l'algorithme, puis trace l'algorithme complet corrigé.

**EXERCICE 19** Caisse d'épargne du quartier

Mama Rose dépose 5 000 FCFA chaque semaine dans une caisse d'épargne. Elle souhaite acheter une machine à coudre coûtant 85 000 FCFA.

1. Écris en pseudo-code un algorithme qui lit le montant de l'objectif (85 000 FCFA) et le dépôt hebdomadaire (5 000 FCFA), puis calcule et affiche le nombre de semaines nécessaires pour atteindre ou dépasser l'objectif, en utilisant une boucle `TantQue`.
2. Trace l'exécution de ton algorithme pour les trois premières semaines (montant épargné et nombre de semaines à chaque tour de boucle).
3. Représente ton algorithme par un algorithme.
4. Mama Rose décide finalement de déposer 10 000 FCFA par semaine. Sans réécrire tout l'algorithme, indique quelle donnée change et donne le nouveau nombre de semaines affiché.

**EXERCICE 20** Type BEPC : bulletin de notes

Au collège bilingue de Bafoussam, le professeur principal veut un algorithme qui lit les trois notes d'un élève (sur 20) en informatique, en mathématiques et en français, calcule la moyenne, puis affiche :

- la décision : « Admis » si la moyenne est supérieure ou égale à 10, « Refusé » sinon ;
  - la mention : « Passable » si  $10 \leq m < 12$ , « Assez bien » si  $12 \leq m < 14$ , « Bien » si  $14 \leq m < 16$ , « Très bien » si  $m \geq 16$  (la mention n'est affichée que si l'élève est admis).
1. Écris en pseudo-code l'algorithme complet (déclaration des variables, lecture des trois notes, calcul de la moyenne, structures alternatives pour la décision et la mention).
  2. Trace l'exécution de ton algorithme pour un élève ayant obtenu 13, 15 et 11 : donne la moyenne, la décision et la mention affichées.
  3. Représente cet algorithme par un algorithme complet (début, lectures, traitement, tests, affichages, fin).
  4. Le proviseur souhaite maintenant que le programme refuse toute note inférieure à 0 ou supérieure à 20 en affichant « Note invalide ». Indique où placer ce contrôle dans ton algorithme et écris la structure alternative correspondante.

## 12 CORRIGÉS DES EXERCICES

## 13 UTILISATION DES STRUCTURES DE CONTRÔLE

### 13.1 Je vérifie mes connaissances

#### **CORRIGÉ** Exercice 1 — QCM : structures de contrôle

1. b. Un algorithme est une suite finie et ordonnée d'instructions permettant de résoudre un problème. Ce n'est pas un langage de programmation (c'est une description abstraite) ni un composant matériel.
2. a. La structure alternative simple s'écrit : Si <condition> Alors <instructions> Sinon <instructions> FinSi. La proposition b correspond à une boucle TantQue, la c à des entrées/sorties.
3. a. La boucle Pour est utilisée lorsque le nombre de répétitions est connu à l'avance (borne de début, borne de fin, pas). La boucle TantQue convient quand la condition d'arrêt dépend d'un calcul imprévisible.
4. b. Dans un algorithme, un test (décision) se représente par un losange. Le rectangle représente une action/traitement, l'ovale le début/fin.

#### **CORRIGÉ** Exercice 2 — Vrai ou faux ? Justifie

1. Vrai. Dans une boucle TantQue, la condition est évaluée avant chaque itération. Si elle est fausse dès la première évaluation, le corps de la boucle n'est jamais exécuté (zéro itération).
2. Faux. Il est indispensable de modifier la variable intervenant dans la condition à l'intérieur de la boucle, sinon la condition reste vraie indéfiniment et la boucle ne s'arrête jamais (boucle infinie).
3. Vrai. Par convention, tout algorithme commence par un ovale « Début » et se termine par un ovale « Fin ».
4. Faux. L'instruction Sinon est optionnelle. On parle d'alternative simple (Si ... Alors ... FinSi) quand il n'y a pas de Sinon, et d'alternative complète quand il y en a un.

**CORRIGÉ** Exercice 3 — Définitions

1. Algorithme : Description finie, non ambiguë et ordonnée d'une suite d'opérations élémentaires permettant de résoudre un problème ou d'obtenir un résultat à partir de données d'entrée. Exemple : La recette de cuisine pour préparer le « ndolé » (entrées : ingrédients ; sorties : plat prêt ; étapes : laver, couper, cuire...).
2. Structure alternative : Structure de contrôle qui permet d'exécuter un bloc d'instructions si une condition est vraie, et éventuellement un autre bloc si elle est fausse. Elle réalise un choix. Exemple : « Si la note  $\geq 10$  Alors « Admis » Sinon « Refusé » FinSi ».
3. Structure itérative (boucle) : Structure de contrôle qui permet de répéter un bloc d'instructions tant qu'une condition est vraie (TantQue) ou un nombre déterminé de fois (Pour). Elle évite la réécriture du code. Exemple : Afficher les nombres de 1 à 10 avec Pour i de 1 à 10 Faire Ecrire(i) FinPour.
4. Algorithme : Représentation graphique normalisée d'un algorithme à l'aide de symboles géométriques (ovale, rectangle, losange, flèches) reliés entre eux, qui montre la logique et le flux de contrôle sans dépendre d'un langage de programmation. Exemple : Le diagramme en losange/rectangles/ovales de l'exercice 8 ci-dessous.

**CORRIGÉ** Exercice 4 — Trace d'exécution

1. Table d'exécution pour  $n = 4$  :

| Étape                | Valeur de i | Valeur de s |
|----------------------|-------------|-------------|
| Avant la boucle      | –           | 0           |
| 1 <sup>er</sup> tour | 1           | 1           |
| 2 <sup>e</sup> tour  | 2           | 3           |
| 3 <sup>e</sup> tour  | 3           | 6           |
| 4 <sup>e</sup> tour  | 4           | 10          |

2. L'algorithme affiche 10. En général, cet algorithme calcule la somme des  $n$  premiers entiers naturels non nuls :  $S = 1 + 2 + \dots + n = \frac{n(n+1)}{2}$ . Vérification : pour  $n = 4$ ,  $\frac{4 \times 5}{2} = 10$ .

## 13.2 Je m'entraîne

**CORRIGÉ** Exercice 5 — Mineur ou majeur

## 1. Algorithme en pseudo-code :

```
Algorithme MajeurMineur
Variable age : Entier
Debut
    Ecrire("Entrez l'âge de l'élève : ")
    Lire(age)
    Si age < 18 Alors
        Ecrire("Mineur")
    Sinon
        Ecrire("Majeur")
    FinSi
Fin
```

## 2. Traces d'exécution :

- Cas 1 : Âge = 15 ans.
  - Lecture :  $\text{age} \leftarrow 15$ .
  - Test :  $15 < 18$  est VRAI.
  - Exécution du Alors : Affichage « Mineur ».
- Cas 2 : Âge = 19 ans.
  - Lecture :  $\text{age} \leftarrow 19$ .
  - Test :  $19 < 18$  est FAUX.
  - Exécution du Sinon : Affichage « Majeur ».

**CORRIGÉ** Exercice 6 — Somme des nombres pairs

## 1. Algorithme (boucle Pour avec pas de 2) :

```
Algorithme SommePairs
Variable i, s : Entier
Debut
    s ← 0
    Pour i de 2 à 50 pas 2 Faire
        s ← s + i
    FinPour
    Ecrire("La somme des nombres pairs de 2 à 50 est : ", s)
Fin
```

Variante possible : Pour i de 1 à 25 Faire  $s \leftarrow s + 2*i$  FinPour.

## 2. Trace partielle (valeurs de i et s) :

| Tour                       | i  | s   |
|----------------------------|----|-----|
| 1 <sup>er</sup>            | 2  | 2   |
| 2 <sup>e</sup>             | 4  | 6   |
| 3 <sup>e</sup>             | 6  | 12  |
| ⋮                          | ⋮  | ⋮   |
| Dernier (25 <sup>e</sup> ) | 50 | 650 |

3. Valeur finale affichée : 650. Vérification : Somme =  $2 + 4 + \dots + 50 = 2(1 + 2 + \dots + 25) = 2 \times \frac{25 \times 26}{2} = 25 \times 26 = 650$ .

### CORRIGÉ Exercice 7 — Le vendeur de crédit téléphonique

#### 1. Algorithme avec TantQue :

```

Algorithme JoursEpargne
Variable epargne, jours : Entier
Debut
    epargne <- 0
    jours <- 0
    TantQue epargne < 30000 Faire
        epargne <- epargne + 2500
        jours <- jours + 1
    FinTantQue
    Ecrire("Nombre de jours nécessaires : ", jours)
    Ecrire("Épargne finale : ", epargne, " FCFA")
Fin

```

2. Vérification par le calcul :  $30\,000 \div 2\,500 = 12$  jours exactement. Après 12 jours :  $12 \times 2\,500 = 30\,000$  FCFA. La condition `epargne < 30000` devient fausse, la boucle s'arrête. L'algorithme affiche 12. ✓
3. Si l'on oublie `jours <- jours + 1` dans la boucle : La variable `jours` reste à 0. La condition `epargne < 30000` ne dépend pas de `jours`, donc l'épargne augmente normalement et la boucle s'arrête au bout de 12 tours. Mais l'affichage final donnera « Nombre de jours nécessaires : 0 », ce qui est incorrect. Si par contre on oublie `epargne <- epargne + 2500`, c'est une boucle infinie car la condition reste vraie éternellement.

### CORRIGÉ Exercice 8 — Le plus grand de deux nombres

#### 1. Algorithme :

```

Algorithme PlusGrand

```

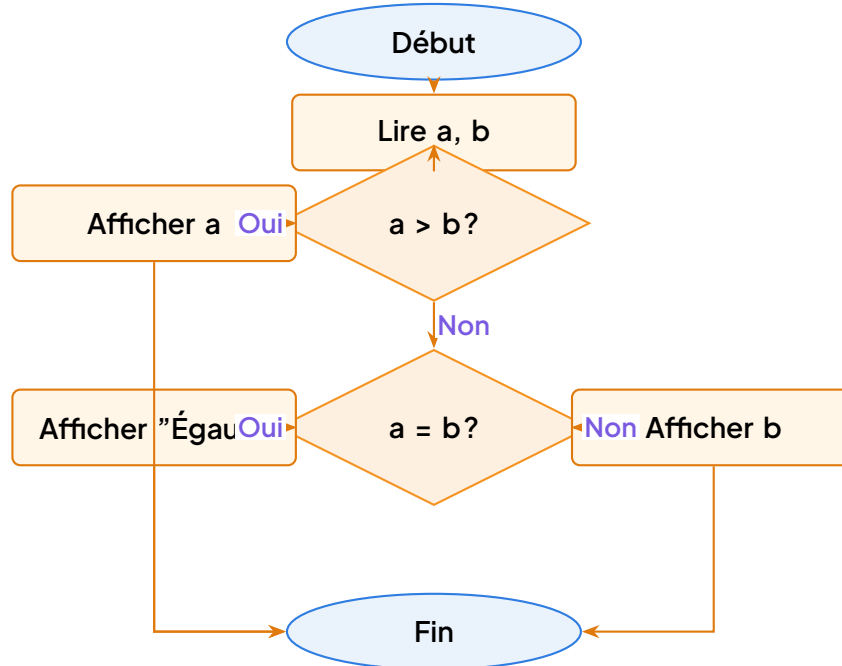
```

Variable a, b : Reel
Debut
  Ecrire("Entrez a : "); Lire(a)
  Ecrire("Entrez b : "); Lire(b)
  Si a > b Alors
    Ecrire("Le plus grand est : ", a)
  Sinon
    Si a = b Alors
      Ecrire("Les deux nombres sont égaux : ", a)
    Sinon
      Ecrire("Le plus grand est : ", b)
    FinSi
  FinSi
Fin

```

Note : La version simple (sans gestion de l'égalité explicite) affiche b si  $a \leq b$ , ce qui est correct pour "le plus grand" mais ne signale pas l'égalité.

## 2. Algorithme :



Algorithme : recherche du plus grand de deux nombres avec gestion de l'égalité

3. Pour traiter correctement le cas d'égalité, il faut ajouter un second test `Si a = b Alors ... Sinon ... FinSi` à l'intérieur de la branche `Sinon` du premier test (comme montré dans le pseudo-code et l'algorithme ci-dessus). Sans cela, l'algorithme considère que b est le plus grand quand  $a = b$ , ce qui est inexact.

### 13.3 Je me prépare au Probatoire

#### **CORRIGÉ** Exercice 9 — Chasse aux erreurs

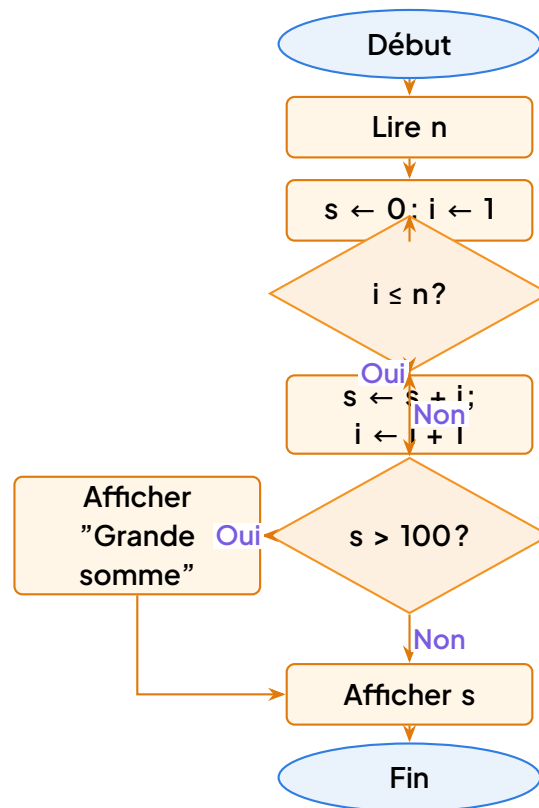
##### 1. Les trois erreurs :

- (a) Boucle infinie (erreur logique) : Dans la boucle `TantQue i <= n Faire ... FinTantQue`, la variable `i` n'est jamais incrémentée. Si  $n \geq 1$ , la condition `i <= n` reste vraie pour toujours → la boucle ne s'arrête jamais.
- (b) `FinSi` manquant (erreur de syntaxe/structure) : L'instruction `Si s > 100 Alors Ecrire("Grande somme")` n'est pas fermée par un `FinSi`. En pseudo-code structuré, tout `Si` doit avoir son `FinSi` (même sans `Sinon`).
- (c) Mauvais symbole dans l'algorithme (erreur de représentation) : Le test `i <= n` est représenté par un rectangle. Or, un test (décision) doit obligatoirement être représenté par un losange. Le rectangle est réservé aux traitements/affectations.

##### 2. Pseudo-code corrigé :

```
Algorithme Somme
Variable n, i, s : Entier
Debut
  Lire(n)
  s <- 0
  i <- 1
  TantQue i <= n Faire
    s <- s + i
    i <- i + 1    // Correction 1 : incrémentation de i
  FinTantQue
  Si s > 100 Alors
    Ecrire("Grande somme")
  FinSi          // Correction 2 : FinSi ajouté
  Ecrire(s)
Fin
```

##### 3. Symbole correct : Le losange (diamant). Algorithme corrigé :



Algorithme corrigé : somme 1 à n avec test sur la somme

**CORRIGÉ** Exercice 10 — Caisse d'épargne du quartier

## 1. Algorithme TantQue :

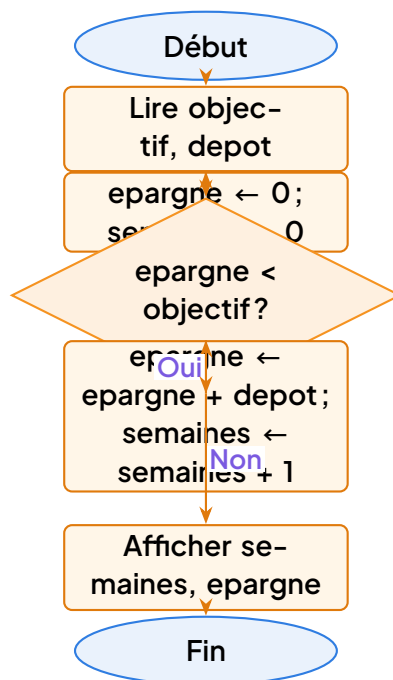
```

Algorithme SemainesEpargne
Variable objectif, depot, epargne, semaines : Entier
Debut
  Ecrire("Objectif (FCFA) : "); Lire(objectif)
  Ecrire("Dépôt hebdomadaire (FCFA) : "); Lire(depot)
  epargne ← 0
  semaines ← 0
  TantQue epargne < objectif Faire
    epargne ← epargne + depot
    semaines ← semaines + 1
  FinTantQue
  Ecrire("Nombre de semaines nécessaires : ", semaines)
  Ecrire("Montant épargné : ", epargne, " FCFA")
Fin
  
```

## 2. Trace pour les 3 premières semaines (objectif=85000, depot=5000) :

| Semaine (tour) | Épargne cumulée | Variable semaines |
|----------------|-----------------|-------------------|
| 1              | 5 000           | 1                 |
| 2              | 10 000          | 2                 |
| 3              | 15 000          | 3                 |

### 3. Algorithme :



Algorithme : calcul du nombre de semaines pour atteindre l'objectif

4. Changement : la valeur de la variable `depot` passe de 5 000 à 10 000 FCFA. Nouveau calcul :  $85\,000 \div 10\,000 = 8,5$ . Il faut 9 semaines (car au bout de 8 semaines :  $80\,000 < 85\,000$  ; au bout de 9 :  $90\,000 \geq 85\,000$ ). L'algorithme affichera 9.

### **CORRIGÉ** Exercice 11 — Type BEPC : bulletin de notes

#### 1. Algorithme complet :

```

Algorithme Bulletin
Variable noteInfo, noteMaths, noteFrancais, moyenne : Reel
Variable decision, mention : Chaîne
Debut
  // Lecture des notes
  Ecrire("Note Informatique (sur 20) : "); Lire(noteInfo)

```

```
Ecrire("Note Mathématiques (sur 20) : "); Lire(noteMaths)
Ecrire("Note Français (sur 20) : "); Lire(noteFrancais)

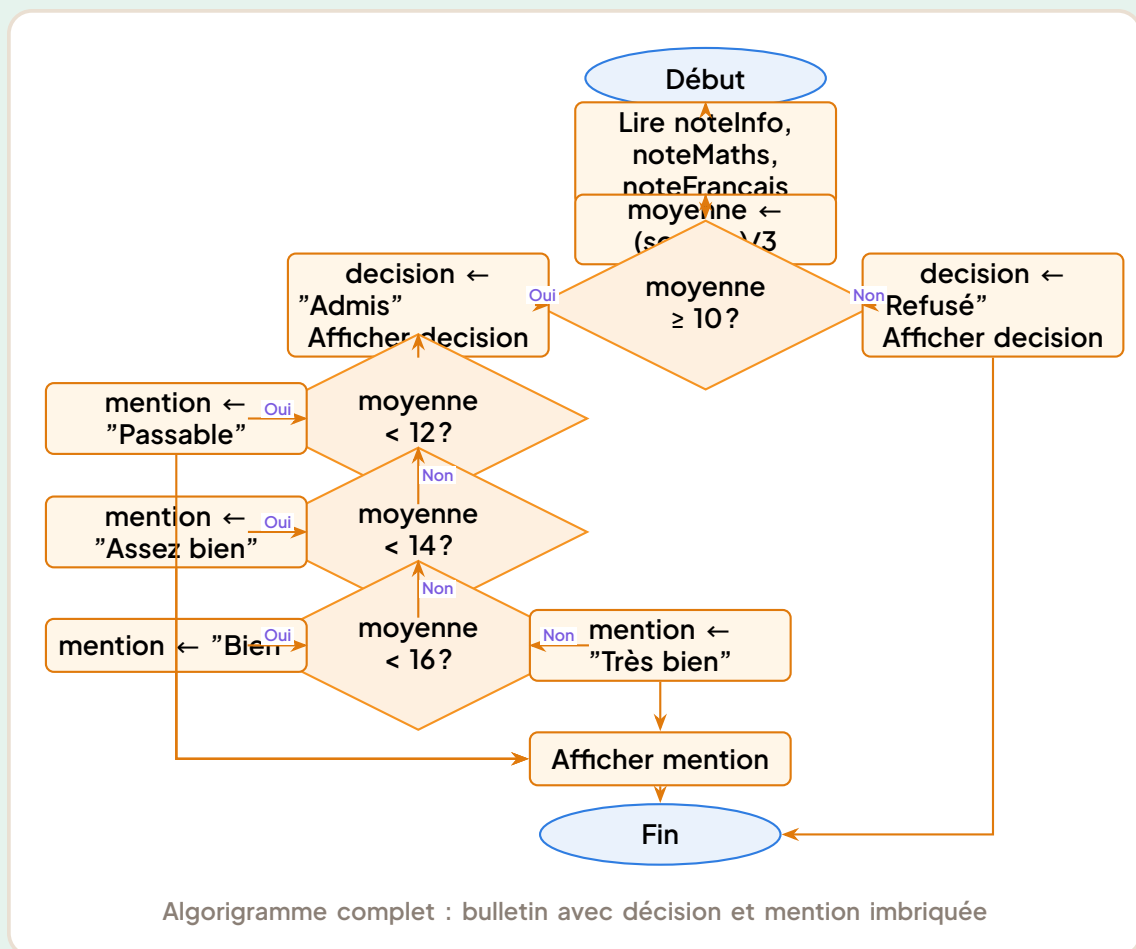
// Calcul de la moyenne
moyenne <- (noteInfo + noteMaths + noteFrancais) / 3

// Décision Admis/Refusé
Si moyenne >= 10 Alors
    decision <- "Admis"
    // Mention (seulement si admis)
    Si moyenne < 12 Alors
        mention <- "Passable"
    Sinon
        Si moyenne < 14 Alors
            mention <- "Assez bien"
        Sinon
            Si moyenne < 16 Alors
                mention <- "Bien"
            Sinon
                mention <- "Très bien"
        FinSi
    FinSi
FinSi
Ecrire("Décision : ", decision)
Ecrire("Mention : ", mention)
Sinon
    decision <- "Refusé"
    Ecrire("Décision : ", decision)
    // Pas de mention pour un refusé
FinSi
Fin
```

## 2. Trace pour notes : 13, 15, 11.

- Somme =  $13 + 15 + 11 = 39$ .
- Moyenne =  $39 / 3 = 13$ .
- Test `moyenne >= 10` :  $13 \geq 10 \rightarrow \text{VRAI} \rightarrow \text{decision} = \text{"Admis"}$ .
- Test mention :
  - $13 < 12$  ? FAUX.
  - $13 < 14$  ? VRAI  $\rightarrow \text{mention} = \text{"Assez bien"}$ .
- Affichage : « Décision : Admis » « Mention : Assez bien »

## 3. Algorithme complet :



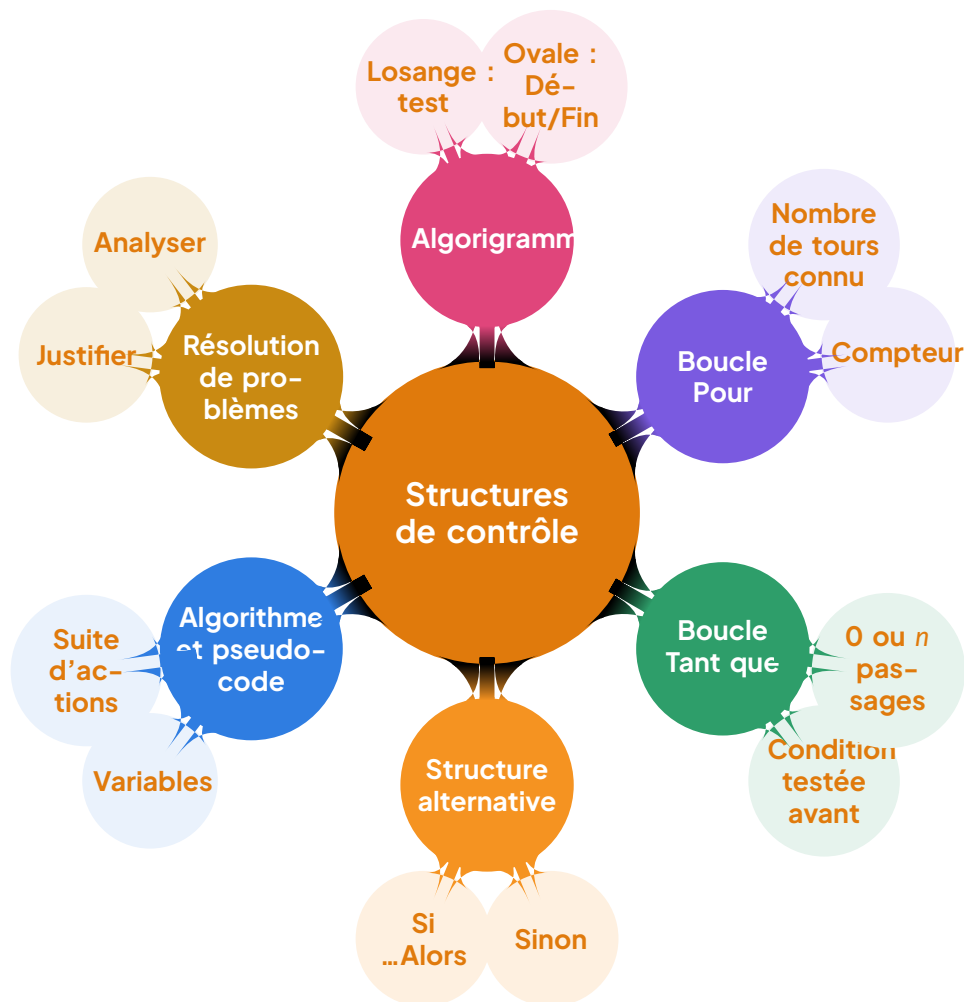
4. Contrôle de validité des notes (0 à 20) : Il faut placer ce contrôle immédiatement après chaque lecture (ou dans une boucle de lecture), avant le calcul de la moyenne. Si une note est invalide, on affiche le message et on arrête (ou on redemande). Structure alternative à insérer après Lire(noteInfo) (et idem pour les deux autres) :

```

Si noteInfo < 0 Ou noteInfo > 20 Alors
  Ecrire("Note invalide")
  // Optionnel : arrêter le programme ou redemander la note
FinSi
  
```

Point de vigilance : L'opérateur « Ou » (ou OU / ||) est nécessaire car la note est invalide si au moins une des deux conditions est vraie (trop petite OU trop grande). L'utilisation de « Et » serait une erreur logique (une note ne peut pas être à la fois < 0 et > 20).

## 14 FICHE BILAN



Carte mentale de la leçon : les structures de contrôle en algorithmique.

### FICHE BILAN L'essentiel à connaître pour l'examen

#### Définitions clés

- **Algorithme** : suite finie et ordonnée d'actions (instructions) permettant de résoudre un problème.
- **Pseudo-code** : langage simple, en français, utilisé pour écrire un algorithme (mots-clés : Algorithme, Début, Fin, Lire, Écrire).
- **Variable** : zone mémoire nommée qui contient une valeur pouvant changer pendant l'exécution.

- **Structure alternative** : permet d'exécuter des actions selon qu'une condition est vraie ou fausse.
- **Structure itérative** (boucle) : permet de répéter des actions plusieurs fois.
- **Algorithme** : représentation graphique d'un algorithme à l'aide de symboles normalisés reliés par des flèches.

Syntaxes à connaître par cœur

| Structure            | Pseudo-code                                           |
|----------------------|-------------------------------------------------------|
| Alternative simple   | Si condition Alors actions FinSi                      |
| Alternative complète | Si condition Alors actions Sinon autres actions FinSi |
| Boucle Tant que      | Tant que condition Faire actions FinTantQue           |
| Boucle Pour          | Pour $i$ de 1 à $n$ Faire actions FinPour             |

Symboles de l'algorithme

- Ovale : Début ou Fin; Rectangle : traitement (calcul, affectation);
- Parallélogramme : entrée/sortie (Lire, Écrire); Losange : test (condition) avec deux sorties Oui/Non;
- Flèches : sens d'exécution.

Méthodes express

- Choisir la boucle : nombre de répétitions connu à l'avance → boucle Pour; répétition selon une condition → boucle Tant que.
- Rédiger un algorithme : (1) comprendre le problème, (2) identifier les données (Lire) et les résultats (Écrire), (3) écrire les traitements, (4) tester à la main avec des valeurs.
- Justifier : expliquer le choix de la structure utilisée et vérifier le résultat sur un exemple concret.

**ATTENTION** Pièges fréquents

Dans un Tant que, une action à l'intérieur de la boucle doit faire évoluer la condition, sinon la boucle ne s'arrête jamais (boucle infinie). Ne pas oublier le Sinon quand les deux cas doivent être traités, ni le FinSi / FinPour / FinTantQue.

**À RETENIR** Checklist avant le BEPC

- ☐ Je sais définir un algorithme et donner un exemple de la vie courante (recette, itinéraire).

- ☐ Je sais écrire un algorithme en pseudo-code avec Algorithme, Début, Fin.
- ☐ Je sais utiliser Lire et Écrire pour les entrées et sorties.
- ☐ Je sais écrire une structure alternative Si ... Alors ... Sinon ... FinSi.
- ☐ Je sais choisir entre la boucle Pour et la boucle Tant que.
- ☐ Je sais écrire une boucle Pour avec un compteur (de 1 à  $n$ ).
- ☐ Je sais écrire une boucle Tant que et éviter la boucle infinie.
- ☐ Je connais les symboles de l'algorithme (ovale, rectangle, parallélogramme, losange, flèches).
- ☐ Je sais dessiner l'algorithme d'un algorithme simple.
- ☐ Je sais tester mon algorithme à la main avec des valeurs concrètes.
- ☐ Je sais rédiger et justifier ma démarche face à un problème concret.

**Signé • OnBuch+**Ludovic A.  
CONFORME AU PROGRAMME OFFICIEL MINESEC**OnBuch+**